Accelerating loop-integral decomposition with exact recurrences and learned Kira strategies

Rushil Saraswat

October 18, 2026

Abstract

We study how exact analytic methods and machine learning can reduce the time needed to decompose loop integrals into master integrals with Kira. The workflow first applies supported analytic identities and independent-loop factorization, then selects the native reduction work that remains. We derive a finite-product reduction for a one-scale massless triangle, including polynomial numerators, and train a cost-sensitive model to choose shared target reductions and sparse seed regions. Training uses complete measured time, including model loading, exact algebra, checks, and recovery. Separate controls distinguish the new learned decision from exact formulas and prior basis preparation. The reserved test contains 58 requests: forty two-loop scattering batches, thirteen independent triangle products through thirteen loops, one symbolic triangle, and four compatibility requests. The final method verifies 58 requests in all three repetitions. On 58 matched requests, it uses 134.119 seconds against 219.277 seconds for the preceding application with the same master indices, a speed factor of 1.63. Full-cost validation enables the new batch model. On the forty native box requests, the added learned choice has time ratio 0.973 against the same workflow with a fixed batch action. Learned row selection remains omitted because its measured improvement is unresolved. The triangle result follows an explicit analytic identity, while learned gains apply to the stated native test domain. These finite tests do not establish a universal speedup for arbitrary connected high-loop integrals.

1 Introduction

Feynman integrals arise when we integrate over the internal loop momenta in higher-order perturbative corrections to scattering amplitudes and particle decay amplitudes. In quantum chromodynamics, these corrections contribute to predictions for cross sections and decay rates. Many integrals in the same calculation have related numerator and denominator structures, and integration-by-parts (IBP) identities allow us to express them in terms of a smaller set of master integrals [1, 2].

We study how to reduce the time needed for this decomposition into master integrals. Kira generates identities and solves the resulting linear system, with existing seed and equation-selection methods that remove some unnecessary work [3, 4, 5]. Exact recurrences can avoid a general linear system for suitable integral structures, while learned choices can reduce the work left for Kira. Throughout this paper, a target is an integral or linear combination requested by the user, and a seed is an index vector at which Kira generates identities. Decomposition determines the coefficients of the master integrals; their numerical or analytic evaluation is a further calculation [1].

Our method combines independent-loop factorization, exact analytic recurrences, and learned seed choices. We also study elimination-row and equation selection to determine where prediction helps. Earlier machine-assisted searches motivate the seed intervention [6, 7], while known analytic one-loop reduction motivates the recurrence stage [8]. We measure the complete application

time, including model use, exact algebra, checks, and unsuccessful attempts. Separate controls distinguish the benefit of a learned decision from the benefit of exact formulas and fixed tuning.

We first define the notation used in this paper and derive the decomposition problem. We then explain the exact stage and the three learned interventions, compare their measured benefits and limits, and test their integration. The latest comparison adds a derived triangle formula and trains a complete-cost model to choose shared native reductions. It tests new two-loop target batches, symbolic triangle kinematics, and independent products through thirteen loops. A separate audit records completion and failure across vacuum, propagator, and scattering sequences at each loop count. It also includes symbolic kinematics. The companion notes preserve historical methods, secondary experiments, and their full results.

2 The reduction problem and the three decisions

An integral family specifies inverse propagators and external kinematics. We use the convention [1, 3]

$$I_{\boldsymbol{a}}(\boldsymbol{x}, d) = \int \prod_{h=1}^{L} \frac{d^d \ell_h}{i\pi^{d/2}} \prod_{j=1}^{N} D_j^{-a_j}, \qquad D_j = q_j^2 - m_j^2 + i0. \tag{1}$$

Here L is the number of loops, ℓ_h is loop momentum h, q_j is a fixed linear combination of loop and external momenta, and m_j is the mass associated with propagator j. The vector $\boldsymbol{x}$ contains squared masses and external momentum invariants. Dimensional regularization uses $d = 4 - 2\epsilon$, where ϵ is the regulator, and $i0$ specifies the Feynman pole prescription. The normalization of the measure is a convention. A positive integer a_j places D_j in the denominator, a negative integer places it in the numerator, and zero removes that factor. The list of N entries includes auxiliary inverse propagators when the physical denominators alone do not span the loop scalar products [1].

Integrals with the same positive-index positions belong to the same sector. A subsector removes some of those positions [3]. At seed $\boldsymbol{\sigma}$, differentiating the integrand with respect to loop momentum ℓ_k gives the identity [2]

$$0 = \int \prod_{h} \frac{d^d \ell_h}{i\pi^{d/2}} \frac{\partial}{\partial \ell_k^\mu} \left(v^\mu \prod_j D_j^{-\sigma_j} \right) = \sum_u A_{\boldsymbol{\sigma},o;u}(\boldsymbol{x}, d) I_{\boldsymbol{a}^{(u)}}. \tag{2}$$

The vector v is a loop or external momentum, and o labels the pair (k, v). The coefficient $A_{\boldsymbol{\sigma},o;u}$ multiplies the integral with index vector $\boldsymbol{a}^{(u)}$. Appendix A expands the derivative and derives the resulting index shifts. Several operators act on the same seed, so seed selection retains or removes a group of candidate equations.

Kira combines IBP identities with symmetry and Lorentz-invariance relations, removes redundant equations, and eliminates integrals in a chosen order. Back-substitution then expresses the requested targets in the remaining master basis [3, 5]. For target T, the result has the form

$$T = \sum_{m=1}^{n_M} r_{Tm}(\boldsymbol{x}, d) M_m, \qquad r_{Tm} \in \mathbb{Q}(\boldsymbol{x}, d). \tag{3}$$

Here M_m is master integral m, n_M is the number of masters, and $\mathbb{Q}(\boldsymbol{x}, d)$ is the field of rational functions with rational coefficients. Equation (3) follows by the exact row operations derived in Appendix A. It determines reduction coefficients; evaluating the masters is a further part of an amplitude calculation [1]. Kira can use symbolic rational arithmetic or repeated finite-field solutions followed by rational reconstruction [9]. We leave finite-field reconstruction unchanged in the native solver. The added recurrence obtains its rational coefficients directly.

Table 1 locates the three learned decisions. Our implementation always obtains the coefficients through exact algebra. If a restricted system does not reach the supplied basis, it repeats the reduction with less restrictive settings, and the timer includes this recovery. Exactness follows from the retained identities and row operations; sufficiency must still be checked for each target, as explained in Appendix A.

Table 1: The three places where a learned decision can change the work. These are the interventions tested in this study.

Decision	Model output	Work affected
Seed selection	Seed region or seed-retention score	Number and structure of identities generated
Row selection	Score for each allowed source row	Intermediate expressions during elimination
Equation selection	Retention score for a seed–operator pair	Symbolic coefficient construction

3 Exact decomposition before learned selection

The form of an integral can remove work before a model makes a decision. Our application first separates independent loop-momentum blocks and uses available scalar and tensor formulas under their stated assumptions, as derived in Appendix G. We add an exact recurrence for scalar one-loop blocks with raised propagator powers. This step extends the application to massive two-point, triangle, box, and pentagon families with a prepared compatible master basis. The remaining blocks retain the learned Kira choices described in the following sections.

The recurrence uses the matrix of masses and external momentum differences defined in Eq. (54). Its dimension equals the number of active denominators, which is at most five in this implementation. Solving the IBP identities for that small matrix expresses a raised power through integrals with a smaller sum of powers. Repeating the step reaches unit-power sectors. Appendix K derives the recurrence, proves termination, and explains why its coefficients can be computed with exact polynomial arithmetic in d. We use this established IBP structure to avoid general equation generation in its supported domain [2, 8].

The application checks the result against the supplied master indices and returns unsupported cases to Kira. It includes the check and any unsuccessful analytic attempt in the total time. This design allows exact formulas and learned choices to contribute to the same decomposition without assigning the analytic saving to ML. The current sparse-seed models remain frozen during the new analytic comparison, and the ML-free control uses the same analytic recurrence. Section 14 reports these separate contributions.

The final input rule limits the new recurrence to direct rational kinematics. Symbolic inputs retain the earlier exact formulas or native Kira path, as tested in Section 15. The symbolic recurrence remains mathematically valid, but its fresh-process setup cost exceeded the native cost in the measured examples.

4 Reducing the cost of the exact stage

The preceding comparison shows that an exact formula can save algebraic work while still spending time on general symbolic setup. We therefore evaluate the finite products in the derived tadpole, one-mass vacuum, and massless propagator formulas directly. Their coefficients remain rational functions of the dimension and the permitted scales. The formulas include the supported quadratic numerators described in Appendix G. Other numerator structures and coupled families pass to the existing reduction methods.

For scalar one-loop blocks, we also combine repeated terms before applying the recurrence in Eq. (58). The coefficients are polynomials in d with exact rational coefficients, so their arithmetic can use integer numerators and denominators throughout. Appendix N derives the combined recurrence. Independent blocks share this method, and repeated identical blocks share intermediate results within one request. No reduction coefficient from an earlier request enters a new answer.

Several raised indices can supply the next recurrence step. Each choice gives a valid decomposition, but the number of intermediate integrals can differ. We train a cost-sensitive tree to select among four strategies: choose the largest positive power, the smallest power greater than one, the first raised index, or the last raised index. A strategy applies at each intermediate integral. The model selects one strategy for the target collection before the exact recurrence begins; it does not predict a coefficient. The eleven inputs describe the number of denominators and targets, the extra positive powers, and the powers at the first five positions. Appendix N gives the features and training objective.

We measure all four strategies on 112 training requests and 56 separate validation requests. Each request contains one or two scalar targets with two through five denominators and at most six extra positive powers. Three repetitions provide a median recurrence time, and all strategies must agree exactly. Validation selects the tree and a minimum predicted saving, with the best fixed strategy as its control. These labels measure recurrence time plus prediction overhead. The reserved application comparison then tests whether the selected policy reduces total time after loading, basis checks, and output verification.

The resulting application first splits independent loop blocks, then attempts a direct formula or the scalar recurrence. It loads the existing learned Kira application when a block needs the remaining exact or native methods. This order reduces setup work on inputs with an immediate analytic decomposition. The retained sparse-seed model keeps its earlier weights. Learned row selection remains omitted, and the equation filter retains its separately tested application. The new recurrence policy adds a different decision to this combination.

Complete application validation also governs deployment of the recurrence model. Section 17 measures that decision on the original validation split, freezes its outcome, and tests twelve fresh requests.

5 Exact triangle reduction and learned target grouping

Some integral structures admit a direct reduction formula. We use such a formula before asking a learned model to choose the remaining native work. The new exact case is a massless one-loop triangle with two lightlike external momenta and one nonzero invariant. Its denominator powers can include a polynomial numerator. Its singular invariant matrix prevents use of the preceding inverse-matrix recurrence in the full triangle sector. Appendix Q derives the coefficient of a single off-shell bubble master from Feynman parameters and expresses it through finite products. This avoids equation generation and elimination for that block. Factorization then applies the formula independently to products of these blocks. The tests through thirteen loops concern these independent products; they do not establish a formula for a general connected thirteen-loop scattering graph.

For the coupled two-loop boxes, several requested integrals often need overlapping equations. We therefore train a model to select how Kira shares the reduction work. The five choices are the preceding application, a common reduction with either of two sparse seed regions, separate reductions for groups of equal numerator rank, and separate reductions for each target. Numerator rank is the sum of the absolute values of negative indices. Each choice uses the same target integrals and checked master basis. Exact algebra determines the coefficients, and an incomplete restricted reduction invokes the existing recovery procedure.

The model uses seventeen numerical features of the denominators and target indices. These include target count, numerator ranks, distances between numerator indices, and how propagators depend on the loop momenta. It predicts the complete cost of each choice. A cost-sensitive tree selects splits that reduce the sum of measured action costs, and bootstrap ensembles provide an additional candidate model [10, 11]. Appendix Q derives the training objective and states all candidate settings. Reference coefficients enter only the independent result checks.

We collect 32 training requests and sixteen separate validation requests from planar and non-planar massless boxes. Their indices have unit positive propagator powers and numerator ranks one through four. We select the model using validation costs, then test its full application cost against the best fixed action at a fresh validation profile. This second check includes loading, prediction, exact reduction, and verification. The model enters deployment only if it saves at least two percent, completes every validation check, and has a request-resampling interval below equal time. The reserved requests remain unused during both decisions.

During development, repeated basis discovery caused avoidable failures on some planar batches. We add a separately checked registry containing only master indices. Two native reductions with different integral orderings must reproduce each other and the development coefficients. The same-basis control receives this registry too, so it separates basis preparation from the new triangle formula and learned grouping. The unmodified preceding application supplies a further practical control. We report basis preparation and benchmark-reference preparation separately from the supplied-basis application timer.

6 Training and comparison design

We use two kinds of labels. Early models learn membership in a recorded sufficient seed set or the next action in a recorded reduction. Such labels describe one valid history and do not establish a minimum number of seeds or operations. Later cost models learn from complete Kira timings for alternative choices. Section 9 uses this second objective to test whether one model can transfer across structures. The notes retain the reverse-construction dataset and its separate label-accuracy tests.

Every reported paired timing ratio uses identical targets, kinematics, and final master bases for its competing methods. The native control disables learned choices in the same exact implementation. A fixed control uses a setting selected before testing and makes no request-dependent prediction. This second control measures what the prediction adds after fixed tuning. The earlier deployed application supplies a further practical comparison. When a study uses a modified executable, “native” means its native decision rules; it does not mean an unmodified Kira installation.

The earlier native comparisons keep d symbolic and assign rational values to the other kinematic variables. The analytic comparisons also include symbolic masses and momentum invariants. Thus the exported coefficients are exact rational functions of d and any remaining symbolic scales. Master discovery and an enlarged-region reference calculation precede the supplied-basis timer. The application receives the master indices, while reference coefficients enter only the final check. We report this preparation cost separately where measured. Adding it to a supplied-basis time describes a two-stage workflow, since the reference calculation already performs a reduction; it does not measure a faster first reduction from scratch.

The new comparisons start a fresh process for each measurement and include imports, model loading, prediction, IBP generation, elimination, exact coefficient comparison, retries, output, and process exit. Three repetitions rotate method order on the same worker for each request. Earlier retained comparisons use their stated timers and repetitions in Table 2. For the seed, equation, and shared-model workloads, we sum the median time of each request and divide by the corresponding control total. The historical row comparison uses a geometric mean, as stated

in Section 7. Appendix E defines this ratio and its resampling interval. A nontrivial batch contains at least one target that reduces to a nonzero expression beyond an unchanged basis element. Failures remain in the coverage counts and receive no completed-time ratio.

7 Seed selection and elimination-row selection

7.1 Learning a sparse seed region

The sparse-seed model limits the distance between a seed's numerator powers and the numerator-power regions of the targets. Its boundary also depends on the seed rank and sector. We fit the boundary by running complete reductions, then train cost predictors to choose among candidate boundaries. Appendix B defines the geometry and objective. Trees and forests provide the candidate cost models [10, 12, 13]; validation determines whether request-dependent prediction improves on a fixed sparse setting.

This retained comparison uses planar and nonplanar two-loop boxes with numerator ranks two through four, unit positive physical indices, rational kinematics, and symbolic d. Twelve training requests and ten validation requests per graph precede the reserved test. Validation retains a fixed boundary for the planar graph and enables a depth-two, 32-tree extremely randomized forest for the nonplanar graph. The 20 reserved requests have seven repetitions per method. All 560 timed reductions pass exact checks, with 5,600 coefficient comparisons. Table 2 shows 22.2% less time than tuned native choices and 3.6% less time than the fixed sparse control. These measurements test new parameter profiles within known graphs; they motivate, but do not establish, cross-family transfer.

7.2 Why the final method retains native row choices

The row model scores the allowed source rows of the current coefficient matrix. Its original network maps 132 inputs to 32 features, pools the available actions, and uses a 64-component hidden layer to produce a probability for each action. The inputs describe the pivot, nonzero entries, matrix size, and sampled cancellations. Exact arithmetic performs the selected row operation, as derived in Appendix A. The network learns the recorded next action through categorical cross-entropy; the notes give its full specification and the derivation of that loss.

In our live Kira comparison, the original row network takes 1.305 times the stock reduction time across 84 exactly checked trials. A later compiled policy trained on algebraic cost gives a geometric mean time ratio of 0.982 with repeat interval $[0.959, 1.003]$. Since this interval includes equal time, that experiment supplies no reliable complete-time gain. We therefore omit learned row selection from the retained method and leave Kira's row choices active. The historical row measurements and their different controls remain in the notes.

8 Selecting equations before coefficient construction

Equation selection makes a finer choice within the seed region. A classification tree scores each seed–operator pair before Kira builds its symbolic coefficients. Its seventeen features describe loop and external-momentum counts, index counts, target number, seed powers, sector membership, distance to a target, and operator index. The score is the weighted positive fraction in the reached leaf, using the tree construction of Ref. [10]. Appendix B.1 defines the features, weights, and threshold rule.

A positive training label means that the equation's set of participating integrals appears in Kira's selected system. This proxy does not identify a unique necessary equation. We collect 137,015 sampled records, including 39,285 positive labels, from 66 training batches. Shared and

graph-specific trees use depths 6, 10, 14, or 18 and minimum leaf populations 4 or 16. Separate validation first retains high-recall candidates, then measures complete reduction cost. A graph uses a filter only when validation gives at least 5% less time than the same tuned calculation with all ML disabled. The selected filter can retain an individual equation or all operators from a retained seed.

The reserved comparison contains 68 matched nontrivial batches from a 74-request grid at one through four loops. Four reference checks fail and two batches are identity-only. Filtering reduces summed median time from 153.937 to 146.586 seconds, or 4.8%, with ratio interval $[0.946, 0.959]$. On the 32 batches where validation enables filtering, time falls from 91.707 to 84.384 seconds, or 8.0%. All 700 timed completions, including the identity-only batches, pass exact checks. Prediction and five recovery attempts remain inside the timer. These results use known graph structures and partly repeated target patterns.

A separate test changes the index batches within the enabled structures while keeping the model fixed. It retains only batches absent from training and validation, with at least one previously unseen index vector. All 24 requests complete, with 144 exact timed reductions and 5,106 coefficient comparisons. Their summed median time changes from 72.295 to 67.478 seconds, or 6.7%, with ratio interval $[0.928, 0.940]$. This index-transfer result supplies the equation-model entry in Table 2; it remains separate from transfer to new graphs.

Table 2: Direct results for the retained local decisions. Times are sums of request medians in seconds. Sparse seeding includes seven repetitions; equation filtering on new index batches includes three. Their workloads differ, so the ratios cannot be multiplied. These are measurements of this study.

Comparison	Control	Learned	Ratio
Sparse seeds / tuned native, 20 two-loop requests	75.843	58.972	0.778
Sparse seeds / fixed sparse, same requests	61.203	58.972	0.964
Equation filter / ML off, 24 new index batches	72.295	67.478	0.933

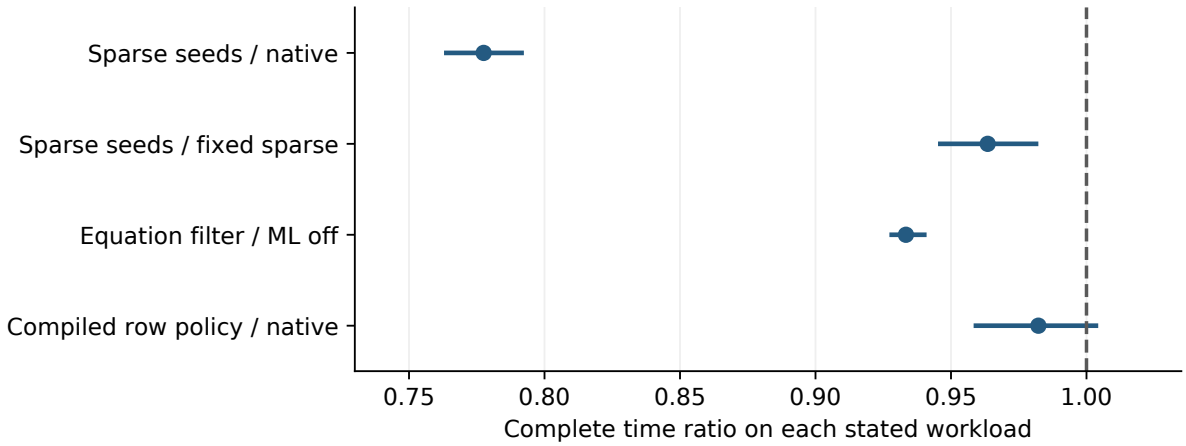


Figure 1: The retained local models reduce time on their stated workloads. Seed selection is compared with both native and fixed sparse choices. Equation selection changes target indices within known structures. Horizontal intervals describe paired repetition variation; the line at one marks equal time. Row selection remains omitted because its later interval includes one.

9 A shared model for transfer across integral families

The preceding gains depend on validated graph domains. We therefore train a shared model from structural quantities, with graph names, stored family signatures, and coordinate-position

identifiers excluded from its inputs. Its 44 inputs summarize the family and targets through loop and external-momentum counts, propagator incidence, masses, positive and negative powers, sector occupancy, and normalized versions of these counts. These features have fixed length when the number of loops or propagators changes. Appendix C specifies the representation and prediction equations.

For each training request, the model predicts the complete cost of ten seed configurations, including changed positive-power bounds, numerator-rank bounds, and Kira's subsector truncation. We fit logarithms of time ratios to native bounds and penalize incomplete attempts. These labels include recovery to less restrictive configurations. A decision margin requires a predicted improvement before changing a common anchor action. The anchor and margin are global; no graph-specific deployment rule enters this new model.

We reuse only recorded training costs and reserve the earlier validation costs for model selection. The distinct-mass vacuum family of Appendix D, nonplanar boxes, pentagons, and loop orders four and above are excluded from both sets. The remaining data contain 74 training and 51 validation requests. Candidate models include nine tree or forest configurations, two regularized linear regressions, and two neural networks. Five-fold validation keeps each ordered graph structure within one fold, and the model selection also considers the separate validation set. Training selects a neural network with two 48-component hidden layers and a margin of 0.05. The network has 5,002 fitted weights and biases. The runtime evaluates the saved weights directly, and its predictions agree with those of the training library within the numerical check tolerance. Appendix C gives the loss, candidate settings, and selection rule.

The reserved physical profile is new. Its 110 requests span the defined one-mass and massive vacuum families, massless and massive two-point families, four-point ladders, boxes with bubble subgraphs, and three additional scattering structures. The declared loop range is one through twelve. This tests both new parameter values in training domains and transfer to the excluded families or higher orders. Appendix D gives the denominators, kinematics, and target construction. Each request has an explicit outcome; a model prediction for an unsupported native input is recorded without calling it a successful reduction.

9.1 Reserved results and loop coverage

The corrected shared model completes 43 matched nontrivial requests from the 110-request grid. The coverage record contains 21 failed reference calculations, 10 failed enlarged-region checks, 0 incomplete method comparisons, and 36 inputs above the native coordinate limit. Identity-only batches number 0. The matched outputs pass 5,976 coefficient comparisons. Table 4 retains every loop order and outcome.

Complete supplied-basis time, summed over request medians, is 63.450 seconds for native choices, 60.094 seconds for the common fixed setting, 57.125 seconds for the shared model, and 59.198 seconds for the preceding application. The learned-to-native ratio is 0.900, with repeat interval [0.867,0.919]. The learned-to-fixed ratio is 0.951, with interval [0.916,0.971]. Relative to the preceding application, the ratio is 0.965, with interval [0.900,0.979]. All ratios include inference, checking, and recovery.

The 15 matched requests from excluded families or higher orders give a learned-to-native ratio of 0.939, with repeat interval [0.897,0.978], and a learned-to-fixed ratio of 0.885, with interval [0.840,0.927]. Their physical groups were excluded from both fitting and model selection.

The model changes the initial action from native bounds on 43 matched requests and from the fixed control on 17. Thus the comparison records actual action changes as well as complete time. Reference discovery across the requested grid takes 1056.4 seconds and the independent checks take 772.2 seconds, including unsuccessful attempts. These costs precede the matched supplied-basis timings. The new fit takes 9.5 seconds and reuses the existing cost labels; this

Table 3: The shared model across physical classes. Ratios use sums of median times on each matched set. Values below one mean less complete time. A class can contain both training-domain and held-out loop orders.

Physical class	Matched	Model/native	Model/fixed
Box with bubbles	4	0.597	0.981
Four-point ladder	4	1.109	1.042
Nonplanar box	1	1.040	1.010
Box with a virtual leg	2	1.092	0.965
Pentagon	2	0.977	0.915
Massive two-point	7	0.957	0.898
Massless two-point	7	0.793	0.988
Distinct-mass vacuum	7	0.861	0.812
One-mass vacuum	9	1.140	0.983

fitting time does not include the earlier cost of generating those labels.

Table 4: Every requested loop order in the corrected-model test. "Reference" includes failed initial or enlarged-region reductions. "Method" means an incomplete matched comparison. A dash means that no complete ratio is available. The last column compares the shared model with native choices on matched nontrivial requests.

Loops	Requests	Matched	Reference	Method	Unsupported	Ratio
1	12	12	0	0	0	1.027
2	14	13	1	0	0	0.990
3	12	10	2	0	0	0.712
4	12	6	6	0	0	0.827
5	12	2	10	0	0	1.172
6	10	0	8	0	2	—
7	8	0	4	0	4	—
8	6	0	0	0	6	—
9	6	0	0	0	6	—
10	6	0	0	0	6	—
11	6	0	0	0	6	—
12	6	0	0	0	6	—

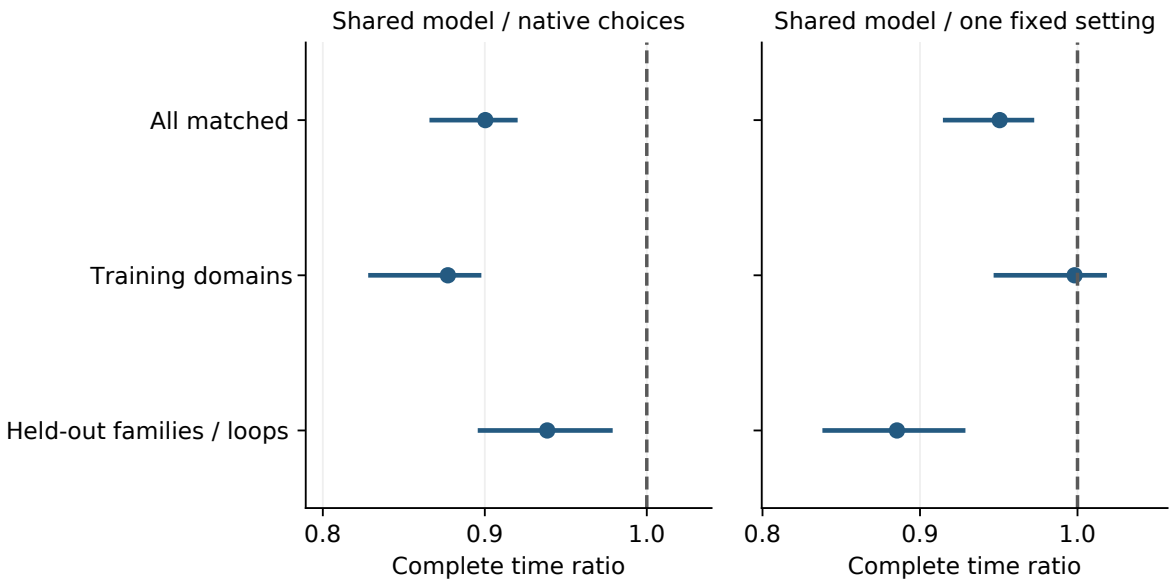


Figure 2: Complete time of the corrected shared model, separated into training domains and physical holdouts. Both controls use the same exact implementation. The fixed control has one setting for all requests. The line at one marks equal time, and the intervals describe paired repeat variation.

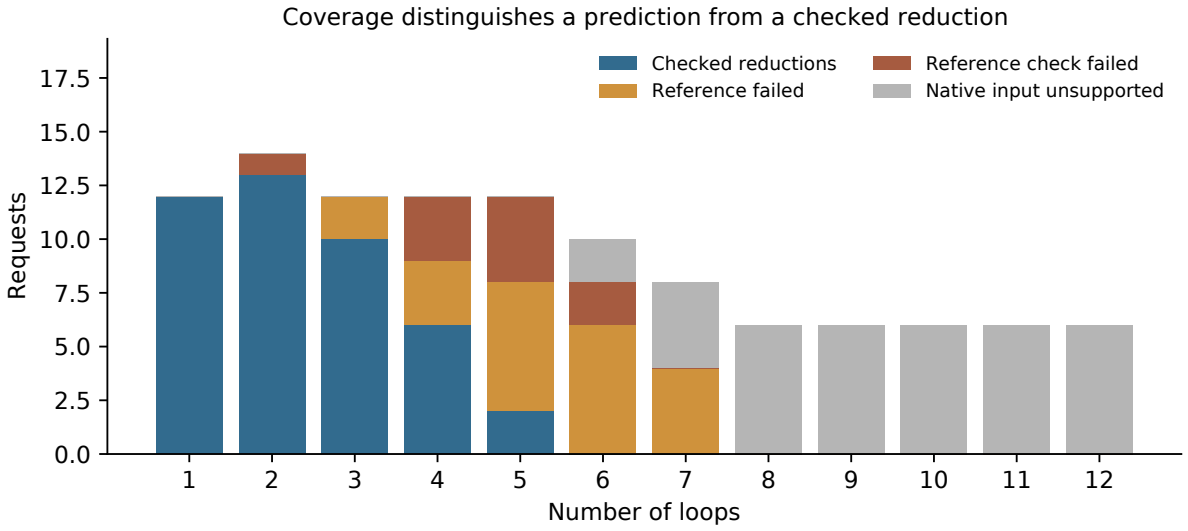


Figure 3: The loop range extends through twelve, while native completion has a smaller measured range. Predictions for unsupported inputs do not count as reductions. Failed reference checks also prevent a timing claim.

The family holdouts test transfer within this finite set of denominator structures, target powers, and rational parameter values. They do not establish performance for arbitrary graph routings, large target ranks, general symbolic kinematics, cuts, or unbounded loop order. The same physical definitions have also been used in earlier project development, although the excluded groups supply no labels or model-selection costs to this new fit. The high-loop failures and native input limits therefore remain part of the result.

10 Reducing preparation failures

The shared-model result separates two costs that require different changes. Its supplied-basis calls complete, while some initial reference calculations or enlarged-region checks fail. We therefore change the preparation workflow and train a second model on individual successful and failed attempts. This comparison includes basis lookup or discovery, both exact reductions, prediction, recovery, and process exit.

The compact workflow first looks for previously checked master indices with the same ordered propagator structure and zero-mass pattern. It reuses these indices while recomputing all coefficients at the new kinematic values. Without a stored basis, it begins with a compact seed region and treats the resulting basis as provisional. A second reduction uses that basis and a different integral ordering, and its coefficients must agree exactly. Kira's supplied-basis check rejects additional masters [5]. Agreement checks the requested reductions; it does not prove that the provisional basis has the globally smallest possible number of masters. Incomplete attempts expand the seed region, and duplicate attempts are removed.

The new model takes the same 44 structural quantities as the neural model, together with the three seed bounds, the truncation flag and level, the presence and size of a supplied basis, and the integral-ordering setting. Its 52 inputs predict log attempt time and a failure score. We fit random forests and extremely randomized forests [12, 13] and select the action with the smallest sum of predicted time and a validation-selected failure penalty. Appendix F defines the outputs and objective. The failure score ranks actions; it is not a calibrated probability.

Training uses 1,405 earlier attempts, including 389 failures, and 58 separate validation requests select the model. The selected random forest has 32 trees of maximum depth 6 and uses a 120-second failure penalty. Fitting and model selection take 3.67 seconds, excluding the earlier cost of collecting labels. Training includes the initial shared-model trial, but excludes the distinct-mass vacuum family, nonplanar boxes, pentagons, and loop orders six and above. The new physical profile supplies no fitting or selection labels.

The reserved comparison contains 33 requests at one through seven loops, using the physical definitions in Appendix D with profile $h = 1237$. It compares expanded discovery and its enlarged-region check with three versions of the compact workflow: native bounds, the frozen neural model, and the cost-and-risk model. Each method receives the same 110-second request deadline and a 60-second cap per native call. Every first attempt is retained, and each initially successful method receives two further repetitions. All four methods use the same 128-bit integral-ordering labels, which Kira supports [5]; this setting changes the internal ordering representation rather than the exact coefficient arithmetic. The 31-coordinate interface limit remains.

The analysis performs 6,375 coefficient comparisons across the repeated and competing outputs. The reused index cache contains 24 earlier checked structures. Their retained discovery and check records cost 341.3 seconds before this experiment; failed attempts during the earlier search add further cost. The compact timings therefore describe use of an existing cache, not cost-free master discovery. In the earlier supplied-basis experiment, 129 neural-model native attempts contain 0 failures. This audit locates the larger failure cost in preparation.

Table 5: Preparation outcomes over the full reserved set. The first-pass timer includes successful and unsuccessful requests. Failed-attempt time counts the unsuccessful native calls within that pass. These totals describe work performed; a shorter failed request is not a successful speedup.

Method	First complete	All 3 complete	First pass (s)	Failed calls (s)
Expanded discovery	17/33	16/33	1041.6	684.2
Compact, no ML	23/33	23/33	757.8	573.1
Compact, neural model	23/33	23/33	869.0	663.4
Compact, cost and risk	23/33	23/33	895.1	677.6

Table 6: Preparation completions by loop order. Each entry requires all three repetitions. The compact columns use the same reuse and checking procedure with the stated decision rule. These results are separate from the earlier supplied-basis coverage in Table 4.

Loops	Requests	Expanded	Compact native	Neural	Cost and risk
1	4	4	4	4	4
2	7	6	7	7	7
3	4	3	3	3	3
4	6	2	3	3	3
5	6	1	4	4	4
6	4	0	2	2	2
7	2	0	0	0	0

Table 7: Complete preparation time on matched requests. Every included method completes all three repeats in the same master-index basis with exact coefficient agreement. The compact/native comparison includes reuse and the changed check; the two ML comparisons isolate prediction within that workflow.

Comparison	Matched	Time ratio	Repeat interval
Compact without ML / expanded	13	0.305	[0.290,0.329]
Neural model / compact without ML	22	0.982	[0.959,1.021]
Cost and risk / compact without ML	22	1.635	[1.604,1.700]
Cost and risk / neural model	23	1.536	[1.471,1.595]

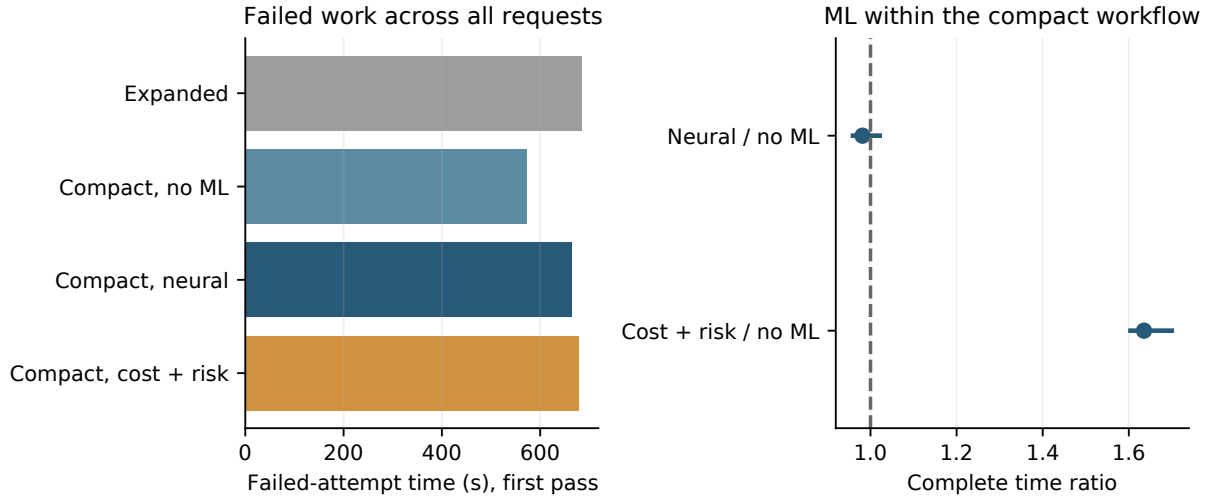


Figure 4: The preparation comparison keeps failed work and the added ML effect separate. The left panel includes failed calls from every first request. The right panel uses only exact matched completions and compares both models with the same compact workflow without ML. The dashed line marks equal time.

Compact preparation without ML compared with expanded discovery gives ratio 0.305 on 13 matched requests, with repeat interval [0.290,0.329]. The neural model within compact preparation gives ratio 0.982 on 22 matched requests, with repeat interval [0.959,1.021]. The cost-and-risk model within compact preparation gives ratio 1.635 on 22 matched requests, with repeat interval [1.604,1.700]. The cost-and-risk model increases time in this broad comparison. The neural preparation result also remains consistent with equal time. We keep the failure model and its records for analysis, while the shared neural model remains the validated supplied-basis default.

The full application first checks whether the restricted scalar massless formula applies. This is a physical condition, independent of measured time. Among the remaining requests, the neural preparation model gives ratio 0.985 on 21 matched completions, with repeat interval [0.963,1.025], relative to compact preparation without ML. These measurements do not meet both conditions for retaining learned preparation: a time interval below one and completion of every request completed by its native control. The default therefore uses native choices for preparation and retains the shared neural model for supplied-basis reduction, where the separate test establishes a gain. This deployment choice follows the candidate comparisons. The reported ratios belong to the frozen methods; they do not provide a new independent timing test of the resulting selector.

10.1 Recovery from an incomplete stored basis

The first compact trial exposed a target-dependent basis mismatch: a basis checked for earlier targets can omit masters needed by a new batch. The revised application discards a rejected cached basis after expanded seeding also fails, rediscovers without that constraint, and checks the result. It also permits an expanded target check when smaller checks fail. All steps remain inside the same 110-second request budget. A focused follow-up uses six fresh requests at two through five loops and keeps the models and index cache fixed. All three repetitions complete on 6/6 requests with ML disabled, 6/6 with the neural model, and 6/6 with the cost-and-risk model. The neural model gives a ratio of 0.783 on 6 matched requests, with repeat interval [0.743,0.797], relative to the revised workflow without ML. The cost-and-risk model gives a ratio of 0.801 on 6 matched requests, with repeat interval [0.772,0.857], relative to the revised workflow without ML. The comparison passes 2,352 coefficient checks. It tests the recovery change on fresh parameters of selected structures and does not replace the broad trial.

11 Exact loop blocks and learned costs through thirteen loops

The next extension changes the calculation around the learned choice. When the active propagators and numerators divide the loop momenta into independent blocks, we reduce each distinct block once and keep the product of its master expansions in factored form. Shared external momenta do not prevent this separation. An active numerator that joins two loop sets prevents their separate reduction. Appendix G derives the factorization and gives exact tadpole, one-mass vacuum, and massless two-point formulas used before native Kira. These routes use mathematical identities and make no learned choice.

The remaining blocks use the same 44 structural inputs and ten seed actions as the shared model. We fit eleven new candidates on all 111 available historical training requests, with 76 separate validation requests. They comprise six forests, two regularized linear regressions, and three neural networks. The forests follow the random-forest and extremely randomized-tree methods [12, 13]. Each forest averages its tree predictions. The cost labels sum the native attempts and their recovery cost, since a checked stored basis permits a single production reduction. An unsuccessful full action receives a penalty. The first selection retains the preceding network with two 48-component hidden layers. After adding the two exact convolution formulas, we compare the fitted candidates again on the historical validation requests that still need native Kira, and

retune the fixed control on the same requests. This final selection precedes the corrected reserved test. The notes give the candidate settings and selection rules.

A stored basis contains integral indices, and Kira recomputes coefficients at the current parameters. During development, combining several alternative preferred master lists allowed different seed actions to return different bases. Their coefficients could not be compared entry by entry. We therefore reduce all stored preferred indices at separate development parameters and retain one checked master set per structure. A production result must use this set; otherwise the application retries with less restrictive seeds. Unknown structures require master discovery and a second exact calculation. These preparation costs remain outside the supplied-cache timing and are recorded separately in the notes.

The first reserved grid contains 79 coupled-family requests and 52 independent-block requests through thirteen loops. It records every failure and native input limit. After the basis correction, we use new parameter profiles for the coupled and product tests. Each product uses one- or two-loop blocks; a total of thirteen loops therefore tests the exact composition and its local learned decisions. It does not test a general irreducible thirteen-loop matrix. Further sets check the derived one-mass vacuum and massless two-point formulas with scalar targets and one quadratic numerator. The notes give the initial grid and all outcomes, while Table 8 gives the corrected application's paired results.

All three corrected methods share the block construction, exact formulas, checked basis, and recovery limits. The native method uses native seed bounds. The fixed method uses the preselected common action. The learned method predicts an action from the block structure. Each timing starts a fresh process and includes prediction, native equation generation, reduction, recovery, output, and exact comparison with an independent reference. Three repetitions rotate method order. Reference preparation is separate. A time ratio uses only requests for which both methods complete all three exact checks; incomplete requests remain in the coverage counts. Analytic-only requests contribute no evidence for ML.

The final validation comparison contains 58 requests that need native Kira and 18 requests handled by the exact formulas. It selects a 48-tree extremely randomized forest with maximum depth 10, with decision margin 0.0. The model weights and decision rule remain fixed throughout the corrected reserved tests.

The corrected grid contains 92 requests. The learned route completes all three exact checks on 91 of them, with 0 first-run completions that lack matching reference coefficients. Across all methods, the experiment records 892 timed attempts and 38012 coefficient comparisons.

Table 8: Corrected application: paired time ratios on completed requests. The numerator is the learned application time; the controls share all exact changes. Times are sums of request medians in seconds. Each row has its own matched set. Analytic rows execute the same formula code and contain no ML inference. These are measurements of this study.

Comparison	Requests	Control	Learned	Ratio
Coupled families / native	18	27.400	22.715	0.829
Coupled families / fixed	18	22.471	22.715	1.011
Independent blocks / native	52	187.116	189.802	1.014
Independent blocks / fixed	52	191.693	189.802	0.990
Vacuum formula / native	11	12.441	11.300	0.908
Vacuum formula / fixed	11	11.735	11.300	0.963
Massless formula / native	10	10.895	10.796	0.991
Massless formula / fixed	10	10.866	10.796	0.994

On requests that actually invoke ML, the time ratio is 0.990, with repetition interval $[0.963, 1.026]$, on 63 matched requests. Against the fixed action on its matched ML requests, the time ratio is 0.991, with repetition interval $[0.972, 1.011]$, on 63 matched requests. The

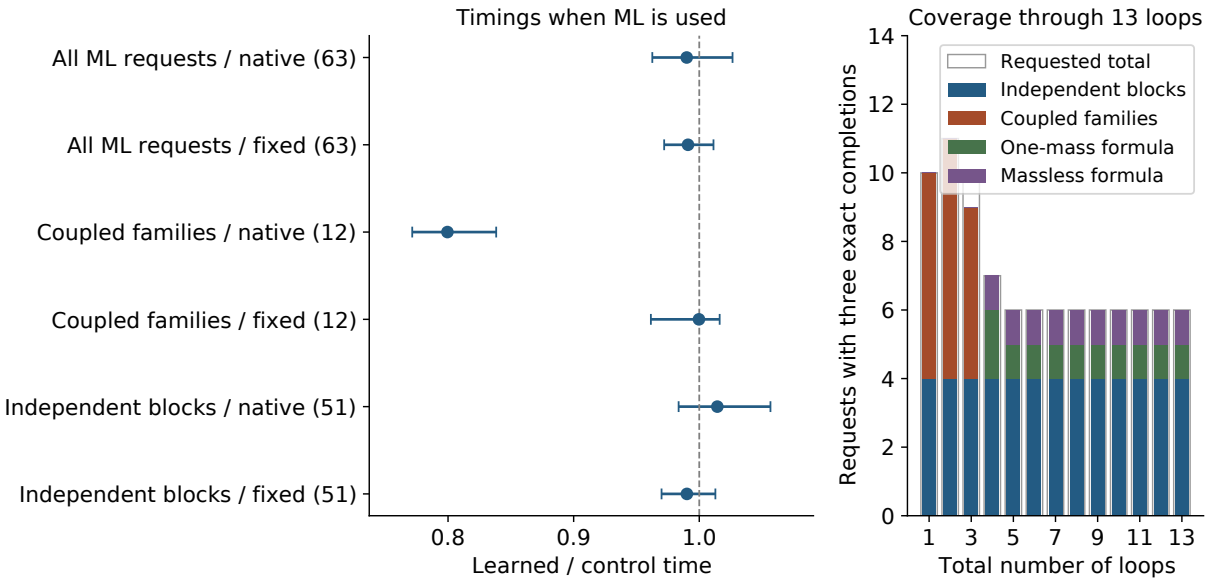


Figure 5: Left: only requests that invoke ML are included, and the controls share all exact changes. The comparisons with fixed choices include equal time within repetition variation. Intervals describe the three repetitions. Counts in parentheses identify matched requests. Right: completion depends on the integral structure. High-loop products contain small independent blocks, while each analytic formula applies to its specified denominator structure. The bar outline gives the requested total. These sets do not establish general coupled high-loop coverage.

comparison with native choices does not resolve a saving beyond repetition variation. The comparison with fixed choices does not resolve a saving beyond repetition variation. These controls isolate the learned seed decision after the exact changes.

The learned method records 45 unsuccessful native attempts among 288 native calls in the timed tests. A completed request includes its failed attempts and recovery in the measured cost. Unverified learned attempts consume 101.255 seconds in addition to the paired completed-request totals. Their outcomes remain in the coverage counts.

Across all timed attempts, the learned application uses 811.364 seconds, native choices use 790.074 seconds, and the fixed choice uses 818.950 seconds. This total includes the unfinished request and all repetitions. The learned application therefore does not reduce total time against native choices on this bounded workload.

In the 63 matched ML requests against the fixed control, the model changes the first native action in 34 requests. The remaining requests retain the fixed action and still pay the prediction cost.

Compared with the initial block application on the matched fresh coupled set, the time ratio is 1.046, with repetition interval $[0.989, 1.085]$, on 16 matched requests. On the separate one-mass vacuum targets with a completed initial block reduction, the time ratio is 0.187, with repetition interval $[0.168, 0.214]$, on 4 matched requests. For the corresponding massless numerator comparison, the time ratio is 0.781, with repetition interval $[0.724, 0.891]$, on 1 matched requests. These changes include exact formulas and revised basis handling, so they do not measure ML alone.

The separate gamma-formula checks cover 14 vacuum requests through thirteen loops and 50 target coefficients. Native Kira confirms 5 of the attempted low-loop requests, with 14 coefficient comparisons; 0 attempted native references do not complete. Higher-loop agreement between the finite-product and gamma expressions checks the derivation and implementation, without establishing a native high-loop timing gain.

The massless numerator checks add 12 requests and 60 gamma comparisons through thirteen loops. Native Kira verifies 3 low-loop requests and 15 coefficients; 0 native references do not

complete. These formulas precede ML and provide no evidence for a learned gain.

12 Choosing baseline seed bounds by complete time

The exact routes in Section 11 leave a native Kira problem whenever no supported formula applies. We compare nine seed settings for this remaining work. The candidates include native bounds, bounds enlarged to contain the supplied masters, and subsector truncations. Two candidates use the least enlargement that admits the required masters under Kira's additional cutoff [5]; Appendix I derives this condition. We fit cost-sensitive trees from 31 training requests and use 20 separate requests for model selection, with all failed attempts and recovery included in the cost.

Validation retains a fixed truncation with an increased numerator-rank bound because the forest's added saving does not reach the declared deployment threshold. Thus this stage supplies an ML-free baseline for the combined method below. The notes preserve its complete model search, measurements, and earlier conservative comparison.

The reserved baseline test requests 65 reductions and verifies 64 in all three repetitions. On matched completions, the selected application takes 209.217 seconds against 213.807 seconds for original native bounds, giving a ratio of 0.979. Its ratio to the preceding application is 0.915; that comparison also includes the revised output handling. The first pass across all requests, including unfinished work, takes 280.860 seconds. The grid spans loop orders one through thirteen through specified formulas and independent products, while coupled native requests reach three loops. The three-loop ladder remains unverified within the resource limits. These are fixed-workflow results and do not establish an added gain from the fitted forest.

13 Combining learned and exact reduction methods

The complete-time comparison above retains a fixed setting for the broad request collection. The earlier sparse-seed model has a separate tested use for nonplanar two-loop integrals with higher numerator powers. We therefore combine these methods and measure their complete costs on fresh inputs. Exact formulas handle their supported families first, and independent loop blocks are reduced separately. For the remaining work, the application can use the preceding setting, a fixed sparse boundary, the trained sparse-seed expert, or the original native bounds. Every route retains exact Kira algebra, the supplied master-basis check, and recovery after an unsuccessful attempt.

13.1 The sparse expert and the complete-time selector

The sparse expert acts before Kira generates equations. It uses the target numerator powers to select the boundary of a seed region, as defined in Section 7 and Appendix B. Its validated input domain contains one target in the specified nonplanar massless two-loop box, with seven unit physical powers and numerator rank two, three, or four. We check the ordered denominator and momentum structure before using this expert. Other blocks use the preceding fixed setting within its training ranges and return to original native bounds outside those ranges. The range check also covers numerator ranks above the broad training collection. This domain restriction permits a useful local model without assigning it an unsupported task.

The expert retains its previously trained 32-tree model with maximum depth 2. Its 12 original training requests and 10 model-selection requests precede this experiment. The model predicts log-time ratios for candidate sparse boundaries from 68 inputs: 27 family and target summaries, 40 positive- and negative-power coordinates, and the largest number of negative entries. Tree evaluation uses the saved thresholds and mean leaf values, following the earlier extremely randomized forest construction [13]. Appendix J gives the candidate boundaries and the fresh target

batches. The current comparison includes feature calculation, prediction, seed enumeration, input preparation, and every recovery attempt.

We then fit an outer cost model from new complete reduction times. It receives the 51 structural and master-basis inputs of Appendix H, together with the two auxiliary numerator powers for this ordered family. Its four actions are the four routes listed above. The training cost includes the inner expert's prediction when that route uses it. We use the direct cost split of Eq. (39), so the outer model learns which complete route takes less time.

The new collection has 16 training requests and 8 validation requests with separate rational kinematics. Each training action is measured once and each validation action twice. We compare one tree or sixteen trees, maximum depths one through five, and minimum leaf sizes two, four, or seven. Validation first compares the outer prediction with the best constant route. It then compares the resulting application with the best ML-free candidate. Each deployment comparison requires at least 1.5% less validation cost and no increase in unverified requests. The fixed sparse candidate retains the boundary selected in the earlier expert study. Current validation compares that candidate with the preceding setting and original native bounds to select the ML-free control. The selected control is a fixed sparse-seed boundary. Validation enables the outer 1-tree model, with maximum depth 2 and minimum leaf size 4. The selected validation cost is 52.826 seconds, compared with 55.917 seconds for the ML-free control. These model-selection results precede the reserved test.

13.2 Fresh complete-workflow comparison

The reserved comparison contains twenty-four nonplanar target requests and ten compatibility requests. The nonplanar requests cover every distribution of numerator ranks two through four across the two auxiliary coordinates at two new kinematic profiles. Eight of these requests use index vectors absent from the new outer-model training and validation collection. The earlier inner expert has a separate training history, which includes some of these added shapes. The ten compatibility requests include native coupled families through three loops, independent products through thirteen loops, and a thirteen-loop analytic case. We freeze the expert weights, outer model, domain checks, and decisions before testing. Each request uses three repetitions with rotated method order on one worker. The preceding application runs through its actual entry point. The fixed control disables both learned predictions, while all routes retain exact coefficient checks and the same native resource limits.

The integrated application verifies 34 of 34 requests in all three repetitions, and invokes learned prediction on 24. For 34 matched requests, the integrated method takes 156.290 seconds and the preceding complete-time application takes 161.592 seconds. This gives 3.3% less time, with ratio 0.967 and repetition interval $[0.950, 1.003]$. For 24 matched requests, the integrated method takes 136.956 seconds and native seed bounds takes 225.338 seconds. This gives 39.2% less time, with ratio 0.608 and repetition interval $[0.584, 0.640]$. For 24 matched requests, the integrated method takes 136.956 seconds and the validation-selected ML-free control takes 143.079 seconds. This gives 4.3% less time, with ratio 0.957 and repetition interval $[0.925, 0.998]$. The comparison with the selected ML-free control resolves an added saving from learned decisions beyond the measured repetition variation. For 8 matched requests, the integrated method takes 52.876 seconds and the validation-selected ML-free control takes 51.685 seconds. This gives 2.3% more time, with ratio 1.023 and repetition interval $[0.974, 1.109]$. For 8 matched requests, the integrated method takes 52.876 seconds and the preceding complete-time application takes 48.131 seconds. This gives 9.9% more time, with ratio 1.099 and repetition interval $[1.066, 1.194]$. These added index patterns expose a remaining transfer limit for the outer selector.

Across all recorded attempts and repetitions, the integrated method takes 476.353 seconds, compared with 493.287 seconds for the ML-free setting and 487.815 seconds for the preceding application. The 128 new training and validation measurements take 1033.449 timed seconds;

Table 9: Complete hybrid comparison. The first-pass time includes one attempt at every requested calculation. Ratios use sums of request medians on matched exact completions. The fixed setting is selected from the three declared ML-free candidates on validation data.

Method	Verified	First pass (s)	Integrated/method	Failed calls
Preceding application	34/34	162.721	0.967	0
Native bounds	34/34	243.584	0.636	0
Selected ML-free control	34/34	165.651	0.961	48
Integrated method	34/34	159.543	1.000	0

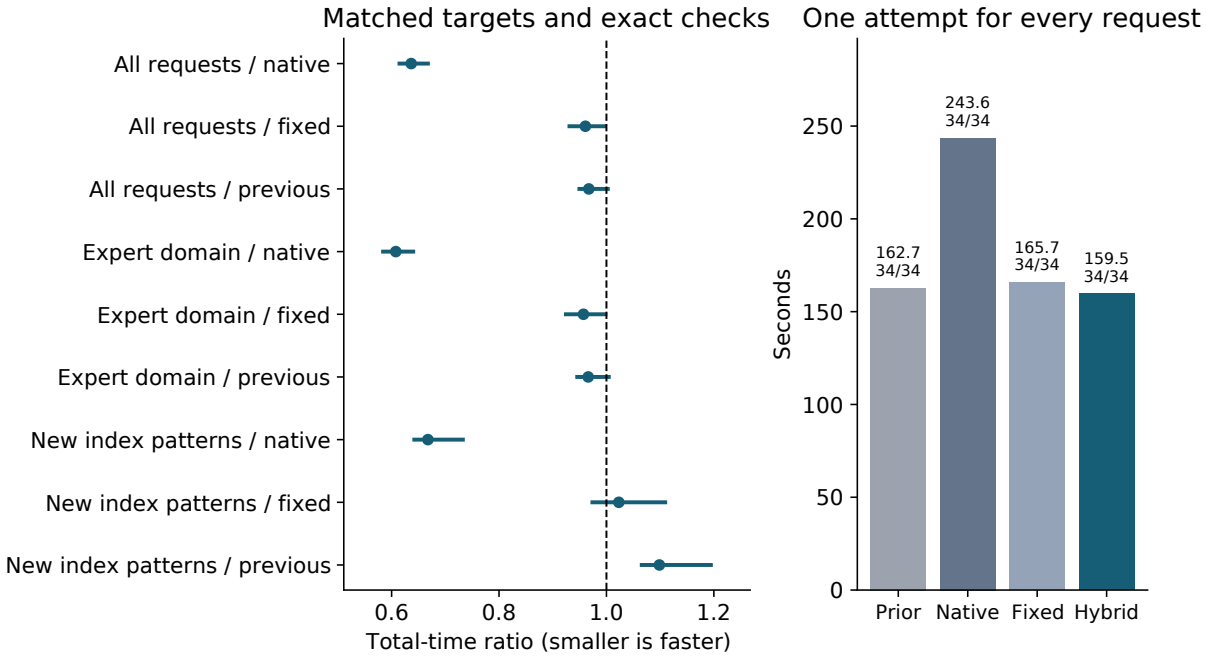


Figure 6: The left panel gives complete-time ratios for all matched requests, the sparse-expert domain, and the additional index patterns absent from outer-model training. The fixed control isolates the added learned decision. The line at one marks equal time, and horizontal bars show paired repetition variation. The right panel includes one attempt for every request and labels verified coverage. Prior and Hybrid denote the preceding and integrated applications.

outer-model fitting takes 12.241 seconds. Independent reference preparation for the reserved requests takes 908.971 seconds outside the application timers. The earlier expert training and checked master-basis preparation are additional prior costs.

The comparison measures use with an existing checked basis and includes model loading, prediction, equation generation, algebra, exact checks, and recovery. The direct learned result applies to the specified nonplanar family and numerator range. The compatibility checks verify the retained routes on their stated families; the broader baseline comparison above remains a separate experiment. The application keeps native row and equation choices. These finite tests establish no globally fastest reduction or coverage of arbitrary high-loop scattering.

14 Combined decomposition with the analytic recurrence

The initial analytic-enabled comparison measures the added recurrence while preserving the existing learned models. It contains 16 rational-kinematics polygon requests, two fully symbolic two-point requests, 12 independent bubble products covering loop orders two through thirteen, eight nonplanar two-loop requests, and three further compatibility requests. Appendix L specifies the denominators, targets, and parameter choices. Each method starts in a fresh process, and three repetitions rotate the method order on the same worker.

All four methods use the same independent-block factorization and share identical blocks within a request. The native control receives the same prepared master indices and uses the existing exact block formulas, but it generates native IBP equations for the newly supported one-loop blocks. The exact fixed control uses the new recurrence and disables learned choices. The preceding application runs unchanged with its earlier master-index cache. Its timer includes discovery and a second check when that cache lacks a structure. The matched native control receives the new prepared indices, which separates the recurrence benefit from this cache extension. Thus the native comparison measures the combined analytic and learned change, while the exact fixed comparison isolates the current learned contribution.

Table 10: Complete decomposition times, summed over per-request medians. Speedup is native-control time divided by combined time. The last column is combined time divided by exact fixed-control time. Ratios below one in that column favor learned selection. Only requests verified in all three repetitions enter a paired sum.

Collection	N	Combined (s)	Native (s)	Speedup	ML/fixed
Scalar one-loop polygons	18	14.815	20.862	1.41	0.991
Independent products	12	4.524	9.522	2.10	1.063
Coupled ML cases	8	43.037	71.455	1.66	0.936
Other compatibility cases	3	2.890	2.918	1.01	0.978
All requests	41	65.266	104.757	1.61	0.958

The combined application completes 41 of 41 requests in all three repetitions. The native, exact fixed, and preceding applications complete 41, 41, and 41 requests, respectively. Including every requested first attempt, the combined application takes 68.401 seconds. Independent reference and master-index preparation takes 357.770 seconds and remains outside the supplied-basis timer.

Against the actual preceding application, the combined method takes 65.266 seconds, compared with 103.134 seconds on 41 matched requests, giving a speedup of 1.58. This comparison includes each application's own preparation and recovery inside its timer.

Across all repetitions, the combined application makes 30 native calls, compared with 120 for the matched native control. The exact fixed control makes 48 calls, including 18 unsuccessful attempts before recovery. The combined method has 0 failed native calls. Thus the recurrence removes native work in its domain, and the retained learned selector avoids the failed sparse

attempts observed with the fixed choice.

For the requests that use the new recurrence, the combined application uses 36.4% less time than the matched native control. The time ratio is 0.636, with repetition interval $[0.608, 0.748]$.

For the requests that use the new recurrence, the combined application uses 0.7% more time than the exact fixed control. The time ratio is 1.007, with repetition interval $[0.787, 1.208]$. This interval includes equal times, so this comparison does not resolve a saving beyond the observed repetition variation.

For the coupled nonplanar requests, the combined application uses 39.8% less time than the matched native control. The time ratio is 0.602, with repetition interval $[0.568, 0.635]$.

For the coupled nonplanar requests, the combined application uses 6.4% less time than the exact fixed control. The time ratio is 0.936, with repetition interval $[0.873, 0.988]$.

For all matched requests, the combined application uses 37.7% less time than the matched native control. The time ratio is 0.623, with repetition interval $[0.599, 0.662]$.

For all matched requests, the combined application uses 4.2% less time than the exact fixed control. The time ratio is 0.958, with repetition interval $[0.869, 1.017]$. This interval includes equal times, so this comparison does not resolve a saving beyond the observed repetition variation.

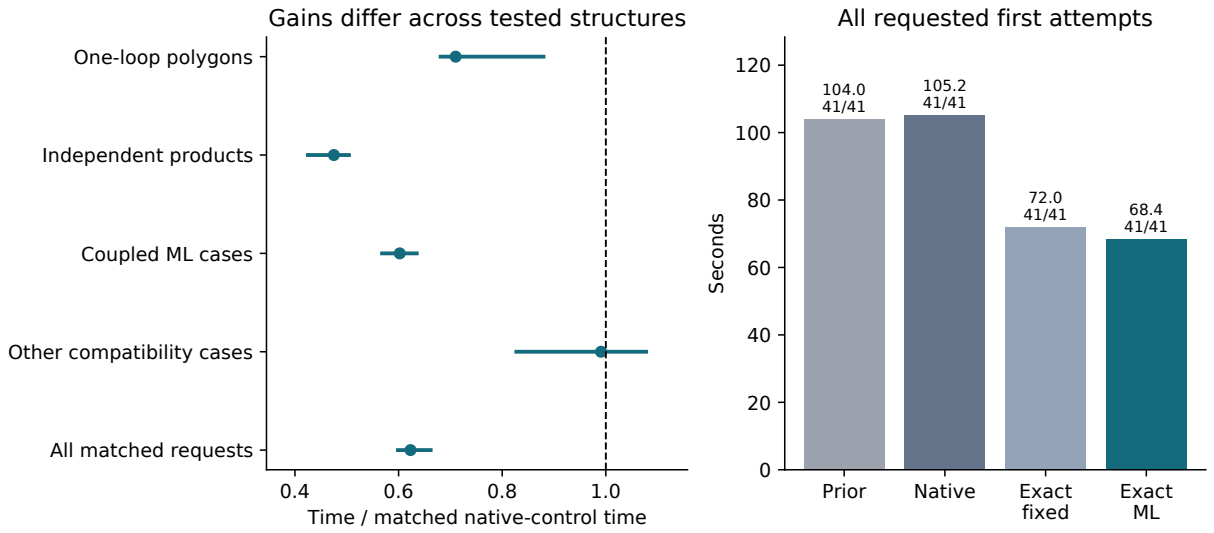


Figure 7: The left panel compares complete verified reductions with the matched native control. The right panel includes every first attempt and gives completed-request counts above the bars. Intervals describe resampling of three timing repetitions on these fixed requests; they do not establish transfer to every integral family.

The analytic improvement comes from the derived recurrence and shared exact subproblems. The models retain their earlier weights and supported domain, so this experiment does not represent new model training. Independent products provide exact tests through thirteen loops, but their connected components remain one-loop integrals. The separate compatibility cases preserve connected native tests through three loops and the existing thirteen-loop vacuum formula. These measurements do not establish a general analytic solution or a universal speedup for connected high-loop scattering.

The two fully symbolic requests expose a cost hidden by the collection average: their analytic-enabled medians are 1.59 and 3.52 times the matched native medians. Each fresh process loads and uses symbolic algebra before it returns the recurrence result. These measurements compare full paths and do not isolate import time. Section 15 tests a separate deployment rule that avoids this setup. The original 41-request measurements remain unchanged.

15 Using native reduction when symbolic algebra costs more

The symbolic regressions motivate a narrower use of the new recurrence. Before entering it, the final application checks whether all masses and scalar products are direct rational values. If an input requires symbolic parsing, the application keeps the older exact formulas where they apply and otherwise uses Kira. This rule leaves the learned models and the existing native settings unchanged. It controls the cost of the new analytic stage; it does not estimate a reduction coefficient.

Two development requests use the massive symbolic two-point family with powers $(4, 1)$ and $(1, 4)$. We enable the rule only when it reduces their summed median time by at least five percent. The reserved comparison then uses powers $(3, 2)$ and $(2, 3)$, two independent products containing those blocks at loop orders two and thirteen, and five compatibility requests. Appendix M specifies these inputs. Each method runs three times in a fresh process, and all four methods pass every exact coefficient comparison.

Table 11: Separate validation of the final input rule. Times are sums of per-request medians in seconds. The initial analytic variant has the new recurrence but lacks the input rule. Native and fixed controls retain the same prepared master indices and factorization.

Collection	N	Final	Initial analytic	Native	Fixed
Symbolic inputs	4	3.333	23.145	3.442	3.461
Compatibility	5	7.919	7.835	10.930	8.294
All requests	9	11.252	30.980	14.371	11.755

For the four symbolic requests, the final method uses 85.6% less time than the initial analytic variant. Its time ratio is 0.144, with repetition interval $[0.098, 0.159]$.

For the five compatibility requests, the final method uses 1.1% more time than the initial analytic variant. Its time ratio is 1.011, with repetition interval $[0.965, 1.105]$. The interval includes equal times.

For all nine requests, the final method uses 63.7% less time than the initial analytic variant. Its time ratio is 0.363, with repetition interval $[0.269, 0.392]$.

Across all nine requests, the final method uses 21.7% less time than the matched native control, with time ratio 0.783 and repetition interval $[0.734, 0.853]$. Against the fixed control, the ratio is 0.957, with interval $[0.913, 1.040]$. This interval includes equal times, so this follow-up does not establish an additional learned saving. The symbolic subset uses the same native branch in both final and native methods; its small timing difference also has an interval that includes one.

All 9 requests complete in every repetition, giving 108 timed reductions and 3,084 exact coefficient comparisons. Independent reference preparation takes 47.230 seconds and remains separate. The first-attempt total for the final method is 12.048 seconds; this total includes every requested input.

The final rule uses native Kira for the four symbolic requests, so their saving relative to the initial analytic variant is an integration improvement. This test compares complete paths and does not separately measure parsing, import, and symbolic arithmetic costs. It is not an additional ML contribution. The earlier 41-request experiment and this nine-request test use different targets and remain separate comparisons. We do not combine their times into a new overall speedup or claim that every symbolic family benefits from this rule.

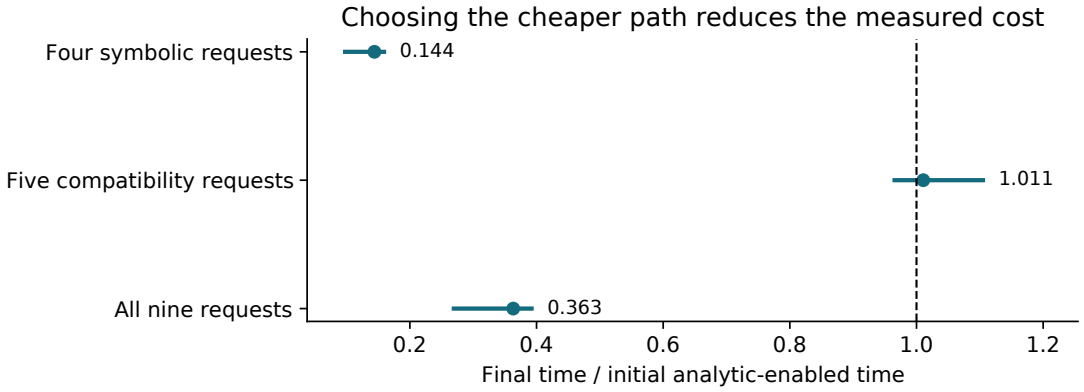


Figure 8: The input rule bypasses symbolic recurrence work where native reduction is cheaper. Values below one favor the final rule. Intervals resample three repetitions on these fixed inputs. Compatibility requests check that other application branches remain available.

16 Complete times and coverage through thirteen loops

The initial fast-implementation comparison tests the direct finite products, faster exact recurrence, and newly trained recurrence strategy together with the retained sparse-seed model. The preceding deployed application is the main practical control. A matched native control retains the earlier exact formulas and block splitting but uses native Kira decisions for the remaining work. The fast fixed control uses the new exact implementation and fixed choices. Thus the native control already contains acceleration, and its timing is not the time of stock Kira on every target.

The reserved set contains 65 requests, including analytic vacuum and propagator families at every loop count through thirteen, distinct triangle products, rational scalar polygons, symbolic bubbles, and coupled native cases. Appendix O defines the targets and parameter profiles. Each method starts a fresh process three times per completed request. The timer includes loading, prediction, exact reduction, output, and the independent coefficient check. Prepared master indices and reference calculations remain separate. Table 12 compares only requests that complete every repetition for both methods.

Table 12: Complete application times in the new reserved test. Times are sums of per-request medians in seconds. Each row uses the same matched requests for its three controls when all methods complete; any unequal counts are reported below.

Collection	N	New learned	Previous	Native	Fast fixed
Vacuum and propagator formulas	26	5.067	21.963	21.568	4.912
Symbolic one-mass vacua	4	0.750	2.728	2.705	0.791
Distinct triangle products	13	3.604	4.279	52.016	3.414
Scalar recurrence	9	2.733	8.716	15.223	2.727
Nonplanar seed model	8	39.518	38.883	66.601	42.392
Symbolic native inputs	2	1.608	1.269	1.448	1.574
Coupled compatibility	3	3.664	3.737	3.806	3.696
All matched requests	65	56.945	81.574	163.366	59.505

The combined method uses 30.2% less time than the preceding deployed application, with time ratio 0.698 and repetition interval $[0.676, 0.717]$. The combined method uses 65.1% less time than the already accelerated native control, with time ratio 0.349 and repetition interval $[0.338, 0.360]$.

The combined method uses 4.3% less time than the fast fixed control, with time ratio 0.957 and repetition interval $[0.925, 0.986]$. This last comparison isolates request-dependent learned choices

after the exact implementation changes. It does not assign the finite-product or arithmetic savings to ML.

The combined method completes 65 of 65 requests in all three repetitions. The preceding, native, and fast fixed methods complete 65, 65, and 65 requests, respectively. Independent reference preparation takes 432.328 seconds. Across the timed runs, the exact checker performs 12,180 coefficient comparisons. The new method's first-attempt total is 59.453 seconds, including 0.000 seconds spent on unverified or incomplete first attempts. Failed requests remain in these counts and receive no completed-time speedup.

16.1 What the new learned decision adds

The recurrence model selects a strategy before the exact substitutions begin. Validation chooses a depth-5 tree, minimum leaf population 3, and prediction margin 0.10. Its fixed alternative selects the last raised index. The measured validation recurrence total changes from 2.6229 to 2.5000 seconds, including prediction overhead. The reserved application test admits the new model on 22 requests and changes the fixed pivot strategy on 3 requests. These groups describe observed use, while the complete scalar-recurrence group was specified before testing.

For the nine reserved scalar-recurrence requests, the combined method uses 0.2% more time than the same fast recurrence with fixed strategy, with time ratio 1.002 and repetition interval $[0.959, 1.076]$. The interval includes equal times. For the eight nonplanar requests, the combined method uses 6.8% less time than the fixed sparse-seed choice, with time ratio 0.932 and repetition interval $[0.888, 0.961]$. The latter model retains its earlier training. These comparisons separate the new recurrence decision from the retained seed decision.

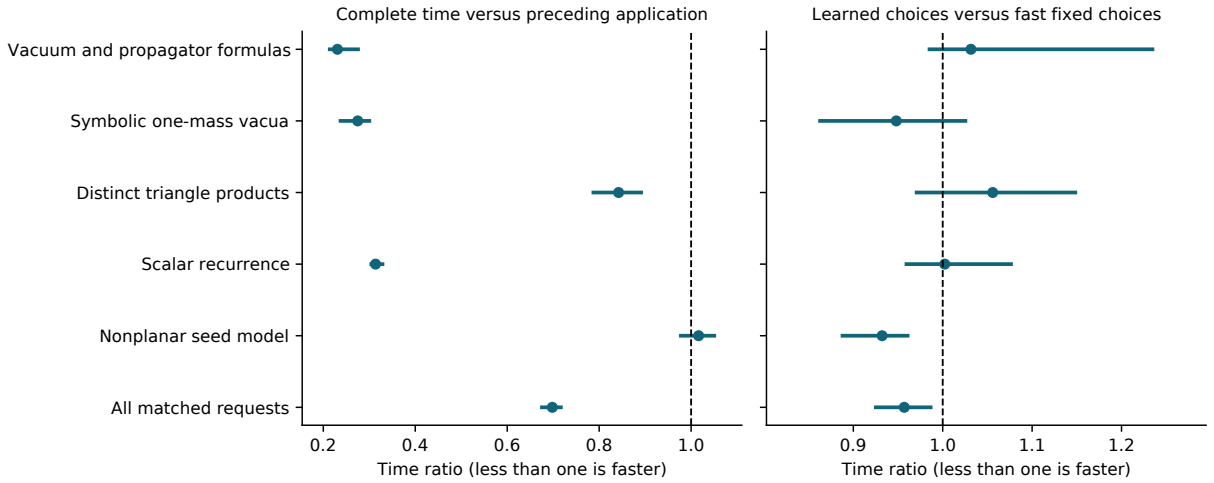


Figure 9: Direct formulas and exact arithmetic reduce complete time, while the fixed control isolates the additional learned choices. Values below one favor the new learned application. Intervals resample the three paired repetitions on the same fixed requests. They do not measure uncertainty over all possible integral families. Formula-only groups use the same exact path in both methods, so their differences measure timing variation.

16.2 Which high-loop requests complete

The timing set deliberately contains analytically supported families. To expose its limits, a separate 110-request audit includes one-mass and all-mass vacua, massless and massive propagators, scattering ladders, and boxes with bubble chains at each loop count through thirteen. It adds scaleless cases, other scattering structures, and symbolic scales. Each application receives twenty

seconds per request. Figure 10 shows the regular loop sequences, and Table 13 includes every audit request.

Table 13: Bounded audit at each loop count. "New" and "Previous" count reported completions within twenty seconds. "Agree" counts paired completions with exact coefficient agreement. The audit has different limits and targets from the timing comparison.

Loops	Requests	New	Previous	Agree
1	12	12	12	12
2	12	12	12	12
3	10	8	8	8
4	7	4	4	4
5	7	4	4	4
6	10	5	5	5
7	7	3	3	3
8	7	3	2	2
9	7	3	2	2
10	7	3	2	2
11	7	3	2	2
12	7	3	2	2
13	10	5	4	4
Total	110	68	62	62

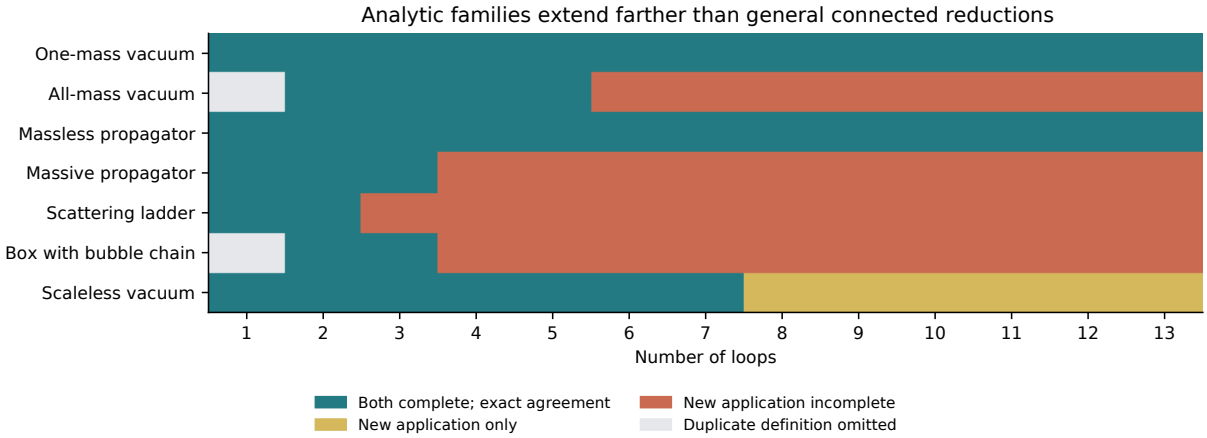


Figure 10: Completion depends on integral structure as well as loop count. The grid shows the new and preceding applications under the same twenty-second audit limit. A failed entry includes timeouts and unsupported inputs. Additional symbolic and scattering requests appear in the complete counts of Table 13.

The 42 incomplete requests comprise 12 time limits and 30 inputs above the native index bound.

The six additional completions are scaleless vacuum requests at eight through thirteen loops. Their exact result is zero by the homogeneous IBP identity derived in Eq. (67). The preceding application rejects their native index count before using this general zero rule. These additional completions come from an exact identity and carry no ML speedup claim.

The analytic families and independent products establish high-loop coverage within their derived domains. General coupled scattering, arbitrary numerator powers, singular kinematics, cut integrals, and symbolic indices require further methods or tests. The native 31-index limit also excludes some complete coupled coordinate systems [5]. Neither prediction nor a shorter failed attempt supplies the missing reduction. The audit therefore records these limits explicitly and does not label its finite collection as all integral types.

17 Selecting the final application by complete time

We selected the new tree using recurrence times. A lower kernel cost alone does not establish a lower application cost, so deployment requires a further check on the original validation split. We run the same 56 validation requests in fresh processes, with prediction, exact reduction, and coefficient comparison inside each timer. The two choices use the same fast implementation and retain the same native seed model; they differ only in the recurrence strategy. Three paired repetitions provide each request's median time. We enable the new recurrence model only when its summed median time is at least two percent lower and its paired repetition interval stays below one. The 65 reserved requests do not enter this decision.

The complete validation total is 13.282 seconds with the recurrence model and 13.408 seconds with the fixed strategy. The ratio is 0.991, with repetition interval $[0.953, 1.015]$. The final application therefore uses the fixed recurrence strategy. The kernel saving does not meet the complete-time deployment rule, so the trained tree remains a recorded candidate. The retained sparse-seed model remains active in its validated coupled domain. This decision leaves all exact formulas and coefficient checks unchanged.

A separate twelve-request test follows the frozen decision. It includes six rational polygons, two distinct-triangle products at two and thirteen loops, a nonplanar box, a symbolic bubble, and two thirteen-loop analytic families. Appendix P specifies their fresh inputs. Table 14 keeps this test separate from the preceding 65-request experiment.

Table 14: Separate reserved test of the final complete-cost rule. Times are sums of medians in seconds. "Candidate" always enables the recurrence model. "Fixed recurrence" retains the native seed model but uses the fixed recurrence strategy.

Collection	N	Final	Candidate	Fixed recurrence	Previous
Scalar recurrence	6	1.708	1.670	1.756	4.125
Distinct products	2	0.614	0.600	0.598	0.791
Nonplanar box	1	3.856	3.719	3.818	3.955
Symbolic bubble	1	0.699	0.737	0.724	0.704
Thirteen-loop formulas	2	0.377	0.323	0.351	1.763
All requests	12	7.254	7.048	7.248	11.338

Against the preceding deployed application, the final method uses 36.0% less time, with ratio 0.640 and repetition interval $[0.612, 0.712]$. Against the always-enabled recurrence candidate, the final method uses 2.9% more time, with ratio 1.029 and repetition interval $[0.981, 1.131]$. This interval includes equal times. Against the fixed recurrence strategy, the final method uses 0.1% more time, with ratio 1.001 and repetition interval $[0.951, 1.118]$. This interval includes equal times.

The final method completes 12 of twelve requests in every repetition. All verified outputs agree with the independent references, with 2,856 coefficient comparisons in this test and 6,072 in the full-cost validation. Reference preparation for the twelve requests takes 73.895 seconds and remains separate. The broad audit in the preceding section belongs to the initial candidate. The final test checks the deployment rule on fresh inputs, while the earlier audit retains its original outcomes and native limits.

18 Complete-time results for exact triangles and target batches

The reserved comparison contains forty native two-loop box requests, thirteen independent triangle products, one triangle with a symbolic scale, and four compatibility requests. Thirty-two box requests use fresh kinematics with the original index patterns, while eight use new index

batches. The compatibility requests retain the earlier vacuum, propagator, and ladder definitions. Appendix Q specifies every collection and the separation between training, validation, and testing.

Each measurement starts a fresh application process and includes loading, model use, exact reduction, retries, output, and the independent coefficient comparison. Three repetitions rotate method order on each request's worker. A first unverified result remains recorded and stops further repetitions for that method and request. Completed-time ratios include only requests verified by both compared methods; the separate completion table retains all requests.

The reserved measurements use two single-core workers on the same Intel(R) Xeon(R) Gold 6240 CPU @ 2.60GHz compute node. We fix that hardware assignment before the reserved tests. The full application validation uses a fresh profile on the same hardware. We preserve an interrupted heterogeneous-worker comparison in the notes; those trials do not determine admission. Each request compares every method on the same worker. The cluster allocation shares the node with other work, and the reported repetition intervals describe variation under those conditions.

Training selects 1 tree with maximum depth 4 and minimum leaf population 2. The full application validation takes 45.942 seconds with the selected policy and 49.598 seconds with the fixed action. Its time ratio is 0.926, with request-resampling interval $[0.853, 0.995]$. The deployment rule therefore enables the new batch model. The separate reserved comparisons below measure the added learned choice after this decision.

The selected fixed action is the wider shared sparse region. The tree changes that action only when its predicted saving exceeds 8% of the fixed cost plus 0.002 seconds. It changes the fixed action on 3 model-selection requests and makes a prediction on 38 reserved requests within its input ranges. Exact formulas and the preceding application handle the remaining structures or inputs outside those ranges.

Within those reserved predictions, the model selects the tighter shared sparse region on 7 requests and the wider shared sparse region on 31 requests. These observed choices show which alternatives contribute to the deployed result.

Table 15: Complete-time comparison with the preceding application supplied with the same checked master indices. Times are sums of request medians in seconds. Each row uses only pairs that pass all three repetitions. A ratio below one means less time.

Collection	Pairs	New	Same-basis control	Ratio
Planar box batches	16	27.549	46.252	0.596
Nonplanar box batches	16	65.740	96.545	0.681
New index batches	8	35.722	44.213	0.808
Triangle products, one to thirteen loops	13	2.540	29.240	0.087
Triangle with symbolic scale	1	0.255	0.642	0.397
Compatibility requests	4	2.313	2.384	0.970
All matched requests	58	134.119	219.277	0.612

Against the preceding method with the same basis, the final method uses 38.8% less time on 58 matched requests. The ratio is 0.612, with repetition interval $[0.604, 0.623]$. The new method is slower on 8 of those individual requests; the largest individual time ratio is 1.157. These per-request outcomes remain visible in the full result record.

Against the unmodified preceding application, the final method uses 67.6% less time on 52 matched requests. The ratio is 0.324, with repetition interval $[0.321, 0.331]$. The matched populations can differ because failed requests remain outside a completed-time ratio. Their totals therefore cannot be subtracted to assign a basis-preparation saving.

Against the fixed batch action over the forty native box requests, the final method uses 2.7% less time on 40 matched requests. The ratio is 0.973, with repetition interval $[0.962, 0.989]$. This

Table 16: Added effect of the new batch decision on native requests. The fixed control has the same exact methods and basis, with the new model disabled. Both methods retain the preceding sparse expert where its action calls for it. These are original measurements of this study.

Collection	Pairs	Final time	Fixed time	Ratio
Planar box batches	16	27.549	28.864	0.954
Nonplanar box batches	16	65.740	71.957	0.914
New index batches	8	35.722	31.725	1.126
All native box batches	40	129.010	132.546	0.973

subtotal isolates the added model on the native workload. The full-workflow comparisons below also retain exact and compatibility requests.

The learned choice takes more time on 19 individual native requests, with a largest request ratio of 2.386. Its aggregate saving therefore does not imply a saving for each input. The recorded native elimination counters total 3,155,328 operations for the learned choice and 3,449,687 for the fixed action. These counters cover the instrumented native algebra; they exclude loading, symbolic checks, and direct-formula arithmetic. The complete timer includes those costs.

Against the same workflow with a fixed batch action, the final method uses 2.5% less time on 58 matched requests. The ratio is 0.975, with repetition interval $[0.961, 0.994]$. Against the same workflow with both learned choices disabled, the final method uses 1.9% less time on 58 matched requests. The ratio is 0.981, with repetition interval $[0.972, 0.999]$. The second control changes the earlier seed choice as well, so its difference measures the combined learned decisions.

Table 17: Completion and first-pass cost over all 58 reserved requests. A verified request must pass all three repetitions. First-pass time sums the first measured application and check cost for every request, including failed work. The paired completed-time ratios appear in Table 15.

Method	Verified	First unverified	First-pass seconds
Final method	58	0	135.430
Fixed batch action	58	0	139.245
Both learned choices disabled	58	0	137.895
Preceding method, same basis	58	0	220.267
Unmodified preceding application	52	6	597.327

The reserved test contains 858 measured outcomes and 14,226 successful coefficient comparisons. The added basis preparation takes 15.747 seconds and supplies 126 coefficient checks across five development requests. Independent reference preparation for the reserved set takes 237.539 seconds. These one-time and benchmark costs remain separate from the supplied-basis totals. The stored master indices contain no solved target coefficients.

Training-data collection, model selection data, and full-cost admission use 1663.451 seconds of summed measured application time. Their independent references add 261.918 seconds. These are separate one-time learning costs. The reserved reduction times measure use of the prepared method and do not imply that training has already paid for itself in a single calculation. Summed process times also differ from elapsed collection time when two workers run concurrently.

The triangle identity has an explicit analytic domain and independent native checks. The learned model has a narrower measured domain: two massless two-loop box structures with rational kinematics and symbolic dimension. The new index batches test transfer within those structures. General connected higher-loop scattering and all integral types remain outside this evidence. A finite candidate search also cannot prove that the selected workflow has the smallest possible runtime.

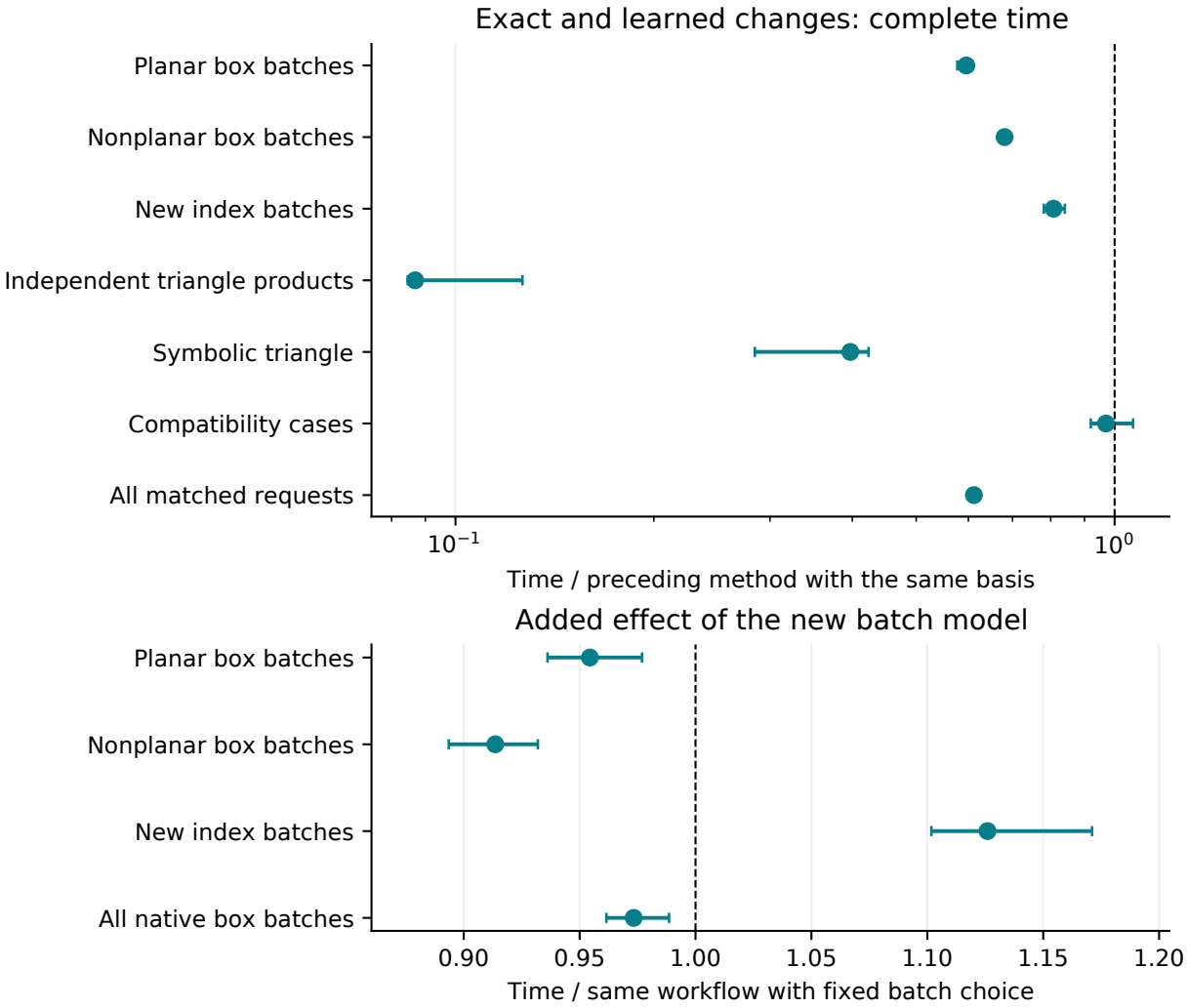


Figure 11: The upper panel measures the combined analytic and learned changes with matched basis preparation. Its logarithmic axis accommodates the exact-triangle saving and the smaller native changes. The lower panel isolates the new batch choice within the native test groups. Horizontal intervals describe paired repetition variation. The line at one indicates equal time; completion counts appear separately in Table 17.

19 Overall interpretation

The latest comparison in Section 18 separates three changes: a checked master-index registry, an exact triangle formula, and a learned choice of target grouping and sparse seed region. The same-basis control isolates the latter two changes. The fixed-action control then measures the added batch model under the same exact implementation, while the control with both learned choices disabled measures its combination with the retained sparse expert. The preceding formula and recurrence experiments remain separate comparisons with their original inputs and controls.

The direct local comparisons establish savings for seed and equation decisions within their measured domains (Table 2). The shared-model comparison tests the additional question of transfer without a family-specific rule (Section 9.1). These are separate workloads and controls, so their ratios do not form a cumulative speedup. Learned row selection remains omitted because its measured gain is not resolved from timing variation. The combined application uses the validated sparse expert together with the exact methods in Sections 11 and 3. The shared model and equation filter retain their separately tested applications, and their gains are not added to the hybrid result.

The preceding complete application combined validated learned choices with exact preparation changes. On 41 matched batches it took 56.461 seconds against 66.045 seconds for native choices, a ratio of 0.855. Its isolated new learned choice changed the seed action in only one batch. We retain that earlier experiment in the notes because its combined gain does not establish a general learned policy. Exact sector-storage changes, scaling reuse, and analytic scalar recurrences likewise remain distinct from learned gains. In particular, the scalar recurrence checks through twelve loops verify a restricted exact formula, not a high-loop ML reduction.

20 Conclusion

The reduction workflow can save time by removing exact work before selecting the remaining algebra. The triangle formula replaces equation generation and elimination within its stated one-scale domain. Independent-loop factorization extends that result to products through thirteen loops. For coupled two-loop boxes, the new model learns whether targets should share a reduction and which sparse seed region should supply the equations. Complete application cost determines whether the learned choice enters deployment.

The latest method verifies 58 of 58 reserved requests in every repetition. Its matched total is 134.119 seconds against 219.277 seconds for the same-basis preceding application, or 1.63 times the speed. Against the fixed batch action, the final method uses 2.5% less time on 58 matched requests. The ratio is 0.975, with repetition interval $[0.961, 0.994]$.

Against the fixed action on the forty native box requests, the final method uses 2.7% less time on 40 matched requests. The ratio is 0.973, with repetition interval $[0.962, 0.989]$. This comparison holds the exact methods and prepared basis constant.

These controls keep the sources of improvement separate. A prepared master-index set avoids repeated discovery, exact formulas avoid whole reductions, and learned choices alter the work left for native Kira. The earlier seed and equation models retain their separate measured gains. Learned row selection remains excluded because its later timing interval includes equal time.

The current result covers an explicit analytic triangle family, independent products, and two learned native graph structures. It does not demonstrate arbitrary connected thirteen-loop scattering. Every failed or unsupported request remains part of the coverage record. Further claims require new physical families, separate training or validation where needed, and complete-time comparisons with equally prepared controls.

A IBP identities, row operations, and exactness

The starting total-derivative identity follows from integration by parts in a region where the integral converges, followed by dimensional analytic continuation [2, 1]. Write $P_{\boldsymbol{\sigma}} = \prod_i D_i^{-\sigma_i}$ and $\partial_k = \partial/\partial\ell_k$. The product rule gives

$$\partial_k \cdot (v P_{\boldsymbol{\sigma}}) = (\partial_k \cdot v) P_{\boldsymbol{\sigma}} - \sum_i \sigma_i (v \cdot \partial_k D_i) D_i^{-\sigma_i - 1} \prod_{j \neq i} D_j^{-\sigma_j}. \tag{4}$$

For $v = \ell_h$, the first factor is $d\delta_{kh}$; for an external momentum it is zero. Completeness of the inverse-propagator coordinates permits us to write $v \cdot \partial_k D_i = u_i + \sum_j v_{ij} D_j$, where u_i and v_{ij} are coefficients fixed by the propagators and kinematics, with the invariant and mass terms included in u_i. Integrating Eq. (4) therefore gives

$$0 = (\partial_k \cdot v) I_{\boldsymbol{\sigma}} - \sum_i \sigma_i u_i I_{\boldsymbol{\sigma} + \boldsymbol{e}_i} - \sum_{i,j} \sigma_i v_{ij} I_{\boldsymbol{\sigma} + \boldsymbol{e}_i - \boldsymbol{e}_j}. \tag{5}$$

The unit vector $\boldsymbol{e}_i$ raises index i by one. This derivation explains how a seed and an operator generate a row of the reduction system.

Let q be a source row and k a pivot column with $A_{qk} \neq 0$. Solving the identity $\sum_j A_{qj} I_j = 0$ for I_k gives $I_k = -\sum_{j \neq k} (A_{qj}/A_{qk}) I_j$. Subtracting A_{rk}/A_{qk} times that zero identity from row r yields

$$A'_{rj} = A_{rj} - \frac{A_{rk}}{A_{qk}} A_{qj}, \qquad A'_{rk} = 0. \tag{6}$$

Every step preserves the represented relation. Repetition until only prescribed master columns remain gives Eq. (3). Removing seeds or equations preserves the validity of the retained identities, but can prevent this elimination from finishing. We therefore require reduction to the prescribed basis and exact agreement with the reference coefficients. Generic kinematics avoid singular pivot denominators; special kinematic limits require their own treatment.

B Sparse seeds and the equation-classifier score

Let $\mathcal{T}$ be the set of target index vectors. For seed $\boldsymbol{\sigma}$, define the extra positive power $D(\boldsymbol{\sigma}) = \sum_i \max(0, \sigma_i - 1)$ and numerator rank $R(\boldsymbol{\sigma}) = \sum_i \max(0, -\sigma_i)$. These definitions count denominator powers beyond one and powers in the numerator. Let $P = \{i : \sigma_i > 0\}$, let h be the number of absent propagators relative to the largest top sector, and let D_T, R_T be the largest target values. The sparse model uses

$$q(\boldsymbol{\sigma}, \mathcal{T}) = \min_{\boldsymbol{a} \in \mathcal{T}} \sum_{i \notin P} \max\{0, \max(0, -\sigma_i) - \max(0, -a_i)\}. \tag{7}$$

For a fixed target, project the nonnegative seed numerator power onto the interval from zero to the target power. The distance is the positive excess in that coordinate. Adding coordinate distances and minimizing over targets gives Eq. (7). To construct a variable-width region, we choose an affine width in the sector deficit, extra powers, and rank. The integer lattice requires its floor, and a maximum with zero prevents a negative width. Applying the same construction to rank and dot allowances gives the restrictions, with fitted parameters $\theta_0, \ldots, \theta_6$,

$$\begin{aligned} q &\leq \max\{0, \lfloor \theta_0 + \theta_1 h + \theta_2 D + \theta_6 R \rfloor\}, \\ R &\leq \max\{0, \lfloor R_T + \theta_3 - \theta_4 h \rfloor\}, \qquad D \leq \max\{0, \lfloor D_T + \theta_5 \rfloor\}. \end{aligned} \tag{8}$$

These inequalities define the search family, and each selected region still requires a sufficiency check. Native outer bounds, retained corners, and admissible target and master seeds also apply.

The tested targets have $D_T = 0$, and $\theta_5 = 1$ remains fixed. The notes give the finite search ranges and fitted vectors.

For B training requests, the boundary search minimizes $J = B^{-1} \sum_b \log[T_b(\boldsymbol{\theta})/T_b^{(0)}]$, where T_b includes reduction, checking, and recovery. Exponentiating gives the geometric mean of the time ratios, which derives the equal-request objective. Failed bounded reductions receive a 300-second training penalty. The subsequent cost predictor compares learned and fixed boundaries before the reserved timing experiment; these training penalties never become successful test times.

B.1 Equation inputs and sampling

The equation classifier uses seventeen inputs. Four describe the family and request:

$$L, \quad E, \quad N, \quad \log(1 + |\mathcal{T}|).$$

Three describe the seed: positive-index count, extra positive power, and numerator rank. Two subtract the batch's maximum extra power and rank from the corresponding seed quantities. Three describe the seed sector: whether it contains a target, and the differences from its target power and rank maxima. Two describe proximity: exact target membership and the minimum sum of absolute index differences. The latter is the defined distance

$$\min_{\boldsymbol{a} \in \mathcal{T}} \sum_i |\sigma_i - a_i|.$$

The operator index and the two overall target maxima complete the vector. These are definitions of the inputs, all available before coefficient construction.

Let N_{cy} be the native number of records of class $y \in \{0, 1\}$ in batch c. The total number of records is $N_c = N_{c0} + N_{c1}$. We sample $n_{cy} = \min(2000, N_{cy})$ records from each class without replacement. Every sampled record receives weight $w_{cy} = N_{cy}/(n_{cy} N_c)$. The total weight of that class is $n_{cy} w_{cy} = N_{cy}/N_c$, so the two classes together give unit batch weight. For a leaf with weighted class totals W_0, W_1, the score is

$$p = \frac{W_1}{W_0 + W_1}. \tag{9}$$

This weighted class fraction is the classifier's score, without a claim of probability calibration. The tree chooses splits using weighted Gini impurity [10]. Expanding $W[1 - p^2 - (1 - p)^2]$, with $W = W_0 + W_1$, gives $2W_0 W_1/W$. A threshold retains an equation if $p \geq \tau$; the whole-seed variant retains every operator if the maximum score over that seed's operators reaches τ. A maximum reaches a threshold exactly when at least one element reaches it, so this variant retains a superset of the source classifier's selected equations. It still requires the sufficiency check above.

C Shared structural model, actions, and fitting

Let n be the number of physical propagators and E the number of independent external momenta. Let B_{ih} be one when loop momentum ℓ_h appears in propagator momentum q_i, and zero otherwise. Table 18 groups the 27 base inputs. Means run over the target batch unless another set is specified. A weighted incidence multiplies the relevant numerator power or extra denominator power by the corresponding row sum of B before summing over propagators.

We add eight ratios: N/n, n/L, massive-entry fraction, coupled-physical-entry fraction, maximum dots divided by n, maximum rank divided by $N - n$, maximum physical loop incidence divided by L, and maximum physical loop degree divided by n. Each denominator is replaced by at least one. Six further features are the mean fractions of indices equal to $-2, -1, 0, 1, 2, 3$. The final three are $\log[1 + L(L + E)]$, $\log(1 + n)(1 + D_T)$, and $\log[1 + \max(1, N - n)](1 + R_T)$.

Table 18: The base structural inputs. Each quantity is computed from the family definition and target indices before Kira generates equations.

Group	Quantities	Number
Family dimensions	L, E, N, n; number of symbolic invariants	5
Propagator structure	Massive-entry count; coupled-physical-entry count; maximum physical row and column sums of B; mean external-momentum incidence over all entries	5
Target batch	Log of one plus target count; number of target sectors	2
Power ranges	Minimum, mean, and maximum extra positive powers and numerator rank	6
Positive indices	Minimum, mean, and maximum number of positive indices	3
Further index limits	Largest absolute index; maximum positive-power sum; maximum number of absent physical denominators	3
Weighted powers	Mean dots on massive entries; mean numerator-weighted and dot-weighted loop incidence	3

These definitions give 44 features. Permuting loop names or entries within the physical and auxiliary groups preserves the pooled counts when the indices are permuted consistently. A change of momentum routing need not preserve them, and identical features need not identify identical reduction problems.

Let (r_0, s_0, d_0) be the target maxima of positive-power sum, numerator rank, and extra positive power. Table 19 defines ten candidate actions. Bounds are nonnegative and r is at least the number t_{top} of positive entries in the largest top sector. Kira's subsector restriction uses [5]

$$s \le \max\{1, t - l + 1, s_{\mathrm{max,sector}}\}, \tag{10}$$

where t is that sector's positive-index count, l is the chosen parameter, and the last term is an explicitly assigned sector limit.

Table 19: The shared model's candidate seed configurations. Changes are relative to the target maxima. A dash disables the additional subsector restriction.

Action	Δr	Δs	Δd_0	l
Native	0	0	0	—
Numerator expansion	0	1	0	—
Tight	-1	-1	-1	—
Reduced dots	-1	0	-1	—
Reduced numerator rank	0	-1	0	—
Expanded	1	1	1	—
Truncated native	0	0	0	$t_{\mathrm{top}} - s_0$
Truncated expansion	0	1	0	$t_{\mathrm{top}} - s_0$
Truncated reduced dots	-1	0	-1	$t_{\mathrm{top}} - s_0$
Relaxed truncation	0	0	0	$t_{\mathrm{top}} - s_0 - 1$

For request c and action a, define the cost label $y_{ca} = \log(T_{ca}/T_{c0})$, with native action 0. Because the denominator is positive and independent of a, and the logarithm is increasing, minimizing the unmodified label selects the action with the least recorded time. Training clips this label to $[-4, 5]$ after assigning failures at least 180 seconds or three times the measured attempt cost, whichever is larger. Clipping limits extreme labels and can merge their order; the validation comparison therefore uses the recorded costs directly.

The neural regression model uses composed affine maps and componentwise activations [14]. The selected network standardizes each feature by its training mean and standard deviation, with

a minimum scale of 10^{-6}. With standardized vector z, its outputs are defined by

$$h_1 = \mathrm{ReLU}(W_1^{\mathsf{T}} z + b_1), \quad h_2 = \mathrm{ReLU}(W_2^{\mathsf{T}} h_1 + b_2), \quad \widehat{y} = W_3^{\mathsf{T}} h_2 + b_3, \tag{11}$$

where $\mathrm{ReLU}(u) = \max(0, u)$ acts on each component. Both hidden vectors have 48 components and the output has ten components, one per action. We fit squared prediction error with a squared-weight penalty of strength 0.1 using the regression implementation of Ref. [14]. For n_{tr} training requests, the corresponding objective is

$$\mathcal{L} = \frac{1}{2n_{\mathrm{tr}}} \sum_{c,a} (\widehat{y}_{ca} - y_{ca})^2 + \frac{0.1}{2n_{\mathrm{tr}}} \sum_j \|W_j\|_F^2. \tag{12}$$

To motivate the first term, assign a formal residual $e_{ca} = \widehat{y}_{ca} - y_{ca}$ the density $p(e) = (2\pi)^{-1/2} \exp(-e^2/2)$. Then

$$-\frac{1}{n_{\mathrm{tr}}} \log \prod_{c,a} p(e_{ca}) = \frac{1}{2n_{\mathrm{tr}}} \sum_{c,a} e_{ca}^2 + \frac{10}{2} \log(2\pi).$$

Removing the constant and adding the stated quadratic penalty gives Eq. (12). This construction motivates a regression objective; the study does not infer a Gaussian distribution of timing errors.

The candidate set also contains trees, random forests, and extremely randomized forests at depths 2, 4, and 7, with at least three samples per leaf; forests contain 48 trees [10, 12, 13]. Two ridge fits use regularization strengths 1 and 10, and the neural candidates have two hidden layers of either 24 or 48 components. Neural fitting uses L-BFGS with at most 500 iterations [14]. The initial implementation exposed an inconsistent scaling between the installed multi-output loss and its gradient. We corrected the loss to match the summed-output gradient, checked it by finite differences, and refitted using the same training and validation inputs. The initial comparison remains in the notes; the corrected model uses a new reserved profile. Candidate convergence warnings remain recorded, and no fit certifies a global optimum.

For every candidate, five-fold cross-validation withholds whole ordered graph structures, with feature scaling fitted within each training fold. The separate validation set selects a single fixed action. We compare that action and native bounds as global anchors, with margins $0, 0.05, 0.12, 0.25$. If b is the anchor and $a_* = \arg\min_a \widehat{y}_a$, the policy changes to a_* only when $\widehat{y}_{a_*} < \widehat{y}_b - \eta$. Exponentiation gives the predicted ratio condition $\exp(\widehat{y}_{a_*} - \widehat{y}_b) < \exp(-\eta)$. We select the candidate minimizing the worse of its grouped-fold and separate-validation ratios to native cost. This rule is fixed before the reserved test. A score below 0.97 permits default deployment; it is evidence from validation, not a guarantee for future inputs.

The selected candidate is the 48-component network, with action 7 from Table 19 (numbered from zero) as anchor and margin 0.05. Training uses only historical training records after the physical holdouts. Their historical timers differ in whether initial imports are included; all new reserved controls use the same complete fresh-process timer. The training record contains every source and model candidate. A failed new seed attempt retries native and expanded bounds with duplicate configurations removed. The shared model never uses target coefficients or measured test time as an input.

D Physical inputs and the loop-order test

The following equations define the physical inputs for our benchmark families. All denominators follow Eq. (1). Let $Q = \sum_{i=1}^{L} \ell_i$. For $L \geq 2$, the vacuum families contain

$$q_i = \ell_i \quad (1 \leq i \leq L), \qquad q_{L+1} = Q. \tag{13}$$

The one-mass case gives only the final entry a nonzero mass; the massive case assigns distinct nonzero masses to all physical entries. The one-loop vacuum is a single massive tadpole. The

two-point families replace the final momentum by $p - Q$, with either all zero masses or distinct nonzero masses. Their single independent external invariant is p^2.

For the four-point families, choose massless external momenta with $p_1 + p_2 + p_3 + p_4 = 0$, and write $P = p_1 + p_2$, $U = p_1 + p_2 + p_3$, $s = P^2$, $t = (p_2 + p_3)^2$. Expanding these squares gives $2p_1 \cdot p_2 = s$, $2p_2 \cdot p_3 = t$, and $2p_1 \cdot p_3 = -s - t$. The physical momenta of the L-loop ladder are

$$\{\ell_i, \ell_i - P : 1 \le i \le L\} \cup \{\ell_i - \ell_{i+1} : 1 \le i < L\} \cup \{\ell_1 - p_1, \ell_L - U\}. \tag{14}$$

We order the loop momenta first, the P-shifted momenta next, then adjacent differences, and finally the two end entries. The box with bubble subgraphs has

$$(q_1, q_2, q_3, q_4) = (\ell_1, \ell_1 - p_1, \ell_1 - P, \ell_1 - U), \qquad (q_{2j+1}, q_{2j+2}) = (\ell_j, \ell_j - \ell_1), \quad 2 \le j \le L. \tag{15}$$

All these four-point propagators are massless. This box family has bubble subgraphs and is a restricted connected scattering class.

The additional massless nonplanar box uses the routing of Ref. [3]:

$$\begin{aligned}(q_1, \ldots, q_9) = (&\ell_1, \ell_2, p_1 - \ell_1, p_2 - \ell_2, p_1 + p_2 - p_3 - \ell_1, \\ &p_1 - \ell_1 + \ell_2, p_1 - p_3 - \ell_1 + \ell_2, \ell_1 - p_3, \ell_2 - p_1 - p_2).\end{aligned} \tag{16}$$

Its first seven entries are physical. Here the outgoing-momentum convention gives $p_i^2 = (p_1 + p_2 - p_3)^2 = 0$, $s = (p_1 + p_2)^2$, and $t = (p_1 - p_3)^2$. Expansion gives $2p_1 \cdot p_3 = -t$ and $2p_2 \cdot p_3 = s + t$. The pentagon uses five massless external momenta with zero sum and physical momenta ℓ, $\ell - p_1$, $\ell - p_1 - p_2$, $\ell - p_1 - p_2 - p_3$, and $\ell - p_1 - p_2 - p_3 - p_4$. Adjacent invariants are $s_{ij} = (p_i + p_j)^2$. Squaring complementary momentum sums gives, for example, $2p_1 \cdot p_3 = s_{45} - s_{12} - s_{23}$. The box with one virtual external leg uses $(\ell, \ell - p_1, \ell - p_1 - p_2, \ell - p_3)$, with $p_i^2 = 0$ for $i = 1, 2, 3$ and $v = (p_1 + p_2 - p_3)^2$; its remaining product is $2p_2 \cdot p_3 = s + t - v$.

There are L loop squares, $L(L-1)/2$ products of distinct loop momenta, and LE loop–external products. Adding these counts gives the complete coordinate number

$$N = \frac{L(L+1)}{2} + LE. \tag{17}$$

This is the representation used by Kira [5]. We append squared momentum differences when their scalar-product coefficient vectors increase the exact rank, until all N coordinates are independent. The candidate order follows loop momenta and then external momenta. Auxiliary indices are nonpositive. The tested interface admits at most 31 coordinates; this is an implementation limit, not an upper bound on the mathematics of IBP. It permits the defined vacuum inputs through seven loops, two-point inputs through six, and four-point inputs through five. Completion remains a separate test.

For the final fresh profile $h = 1213$, choose $m^2_{\text{one}} = 2h+17$, $m_i^2 = 2h+19+6(i-1)$, $p^2 = -2h-29$, $s = -1009-12h$, and $t = -1021-14h$, using only the variables required by each of the six recursive families. All values use a common squared-momentum unit. Additional scattering profiles follow the explicit assignments in Table 20. The independent reference increases the target's positive-power, numerator, and dot limits by one; the check increases the numerator allowance once more and changes integral ordering. Every checked reference and every failure is retained.

Let $a_i^{(0)} = 1$ on the n physical entries and zero otherwise. For $j = 0, \ldots, \min(n, 3) - 1$, set $i_j = 1 + [(h + j) \bmod n]$ and, when $N > n$, $k_j = n + 1 + [(h + j) \bmod (N - n)]$. The dotted batch uses $\boldsymbol{a}^{(0)} + 2\boldsymbol{e}_{i_j}$; the mixed batch uses $\boldsymbol{a}^{(0)} + \boldsymbol{e}_{i_j} - \boldsymbol{e}_{k_j}$. Without an auxiliary entry, the latter has only the extra physical power. The three additional structures use the earlier generator, which puts a second physical dot on the next entry when no auxiliary entry exists. The notes give its full historical target definitions. These are indexed Feynman integrals whose reductions come from exact Kira calculations.

Table 20: Additional scattering kinematics at profile $h = 1201$ for the initial shared-model trial and $h = 1213$ for the corrected-model test. These expressions define the chosen inputs in a common squared-momentum unit.

Family	Invariants
Nonplanar box	$s = -113 - 18h$, $t = -127 - 10h$
Box with virtual leg	Same s, t; $v = -149 - 12h$
Pentagon	$(s_{12}, s_{23}, s_{34}, s_{45}, s_{51}) = (-11, -13, -17, -19, -23) - 20h(1, 1, 1, 1, 1)$

For the retained two-loop sparse-seed test, the planar box has physical momenta $(\ell_1, \ell_1 - p_1, \ell_1 - p_1 - p_2, \ell_2, \ell_2 - p_1 - p_2, \ell_2 - p_1 - p_2 - p_3, \ell_1 - \ell_2)$ and auxiliary momenta $(\ell_1 - p_3, \ell_2 - p_1)$. Its convention is $2p_1 \cdot p_2 = s$, $2p_1 \cdot p_3 = t$, and $2p_2 \cdot p_3 = -s - t$. The nonplanar family is Eq. (16). Both use unit physical powers and numerator ranks two through four on one or two auxiliary entries. The equation-transfer comparison also includes the three-loop vacuum with momenta $(\ell_1, \ell_2, \ell_3, \ell_1 - \ell_2, \ell_2 - \ell_3, \ell_1 - \ell_3)$ and the massive two-point ladder $(\ell_1, \ell_1 - p, \ell_2, \ell_2 - p, \ell_3, \ell_3 - p, \ell_1 - \ell_2, \ell_2 - \ell_3, \ell_1 - \ell_3)$, whose final entry is auxiliary. For the three-loop vacuum the squared masses are $(1, 1, 1, 13, 13, 53 + 2h)$; for the two-point ladder they are $(1, 0, 0, 0, 0, 0, 0, 0)$ on the physical entries, with $p^2 = -1009 - 12h$. The earlier sparse-seed fit uses profiles $h = 831, 832$ for training, $841, 842$ for validation, and $851, 852$ for the 20-request test. Its planar invariants are $(-1009 - 12h, -1021 - 14h)$ and its nonplanar invariants are $(-113 - 18h, -127 - 10h)$. A single-direction numerator carries rank two, three, or four; a distributed numerator splits rank three or four into its ceiling and floor halves on the two auxiliary entries. The equation-filter training uses profiles $511, 512$, validation uses 513, and its new-index comparison uses $605, 606$. The notes preserve the complete finite target enumeration and source records for those earlier experiments.

E Timing ratios and uncertainty

Let T_{cmj} be the elapsed time for request c, method m, and repetition j. For the matched set $\mathcal{C}$, define

$$S_m = \sum_{c \in \mathcal{C}} \operatorname{median}_j T_{cmj}, \qquad R_{m/b} = \frac{S_m}{S_b}, \qquad \text{time saving} = 100(1 - R_{m/b})\%. \tag{18}$$

The sum describes that fixed workload. Subtracting S_m from the baseline total S_b and dividing by S_b gives the saving formula; its speedup factor is $S_b/S_m = 1/R_{m/b}$. A matched request must complete the compared methods and repetitions with exact agreement. We also report all excluded requests, so these ratios are conditional on completion.

For the new test, we resample the three paired repetition indices within each request 2,000 times, recompute its medians and the ratio of totals, and take the 2.5 and 97.5 percentiles. This applies the bootstrap principle [15] to the measured repetition pairs. The interval describes variation on this fixed workload, with limited resolution from three repeats. It does not give a confidence guarantee for unseen integral families or arbitrary loop order. Training, reference discovery, and unsuccessful comparisons are reported separately from matched supplied-basis times.

F Learning attempt cost and failure risk

For recorded attempt i, let t_i be its elapsed time in seconds and f_i be one if it fails and zero otherwise. We define the two regression targets as $u_i = \log \max(0.001, t_i)$ and $v_i = 3f_i$. A tree leaf predicts the mean of each target; averaging the trees gives predictions $\widehat{u}$ and $\widehat{v}$ [12]. The

factor three gives the failure output more weight in the two-output squared-error split objective. For n attempts, that objective is proportional to

$$\sum_{i=1}^{n}[(\widehat{u}_i - u_i)^2 + (\widehat{v}_i - 3f_i)^2]. \tag{19}$$

To obtain the leaf means, differentiate each sum of squared residuals with respect to its constant leaf prediction. The derivatives are $2\sum_i(\widehat{u} - u_i)$ and $2\sum_i(\widehat{v} - 3f_i)$; setting them to zero gives the two sample means. Equation (19) is the chosen fitting objective, without an assumption that every failure has the same physical cause.

The runtime score for candidate action a is defined as

$$C_a = \exp[\mathrm{clip}(\widehat{u}_a, -5, 8)] + \lambda\,\mathrm{clip}(\widehat{v}_a/3, 0, 1). \tag{20}$$

Here $\mathrm{clip}(z, l, h) = \min(h, \max(l, z))$ and λ has units of seconds. The exponential reverses the log-time target, and division by three reverses the failure-label scaling. Their sum defines a time-based ranking with a failure penalty; it is not a calibrated expected-time estimate. Without a supplied basis, candidate actions exclude the three settings that reduce dot or numerator bounds below the native target limits. All ten settings are available when a basis is supplied.

We compare 32-tree random and extremely randomized forests at depths four and six, with at least five training attempts per leaf [12, 13]. Each candidate is evaluated with $\lambda \in \{0, 20, 60, 120\}$ seconds. For each validation request, it chooses one recorded action. Model selection minimizes

$$V = \frac{\sum_c T_{c,a(c)} + 60\sum_c f_{c,a(c)}}{\sum_c T_{c,0}}, \tag{21}$$

where T is the earlier complete action cost including its recovery, f records failure of that action's first attempt, and action zero uses native bounds. Unsuccessful complete validation records receive a 180-second penalty. The denominator is independent of the candidate, so minimizing V minimizes the penalized numerator. The extra 60 seconds deliberately emphasizes avoiding a failed first action even when recovery already contributes to T. The model, penalty, and basis cache are fixed before the reserved comparison.

The new grid contains one target shape per selected family and loop order: dotted targets at odd orders and mixed targets at even orders, as defined in Appendix D. The one-mass vacuum uses orders 1, 2, 4, 5, 6, 7; the distinct-mass vacuum uses 2, 4, 5, 6, 7; each two-point class uses 2 through 6; the ladder uses 1 through 5; and the box with bubbles uses 2 through 5. The nonplanar box, pentagon, and box with one virtual leg add three requests. The cache uses only the master indices from the earlier profile 1201. Stored coefficients never enter prediction or the new reduction. Its structure key distinguishes the ordered momenta and zero versus nonzero mass and invariant assignments. A matching key is only a proposed basis: the new exact calculations must establish target reduction at the new parameters.

Every initially successful method receives three measurements. A paired time ratio includes only requests that complete all three repeats for both methods, use the same master-index set, and agree coefficient by coefficient. Repetition and cross-method checks use exact rational simplification. Different provisional bases, incomplete repetitions, and failed requests remain in the outcome records without a matched ratio. This makes the time comparison conditional on equal completed work.

The focused recovery comparison uses profile $h = 1249$ for the distinct-mass vacuums at four and five loops, the massless and massive three-loop two-point families, the three-loop box with bubbles, and the two-loop ladder. It uses dotted targets at odd orders and mixed targets at even orders. Three repetitions compare the same compact workflow with native bounds, the neural model, and the cost-and-risk model. Model weights and cache entries are unchanged. The revised recovery rule follows the basis rejection observed in the preceding experiment; this follow-up therefore tests that specific correction.

G Independent loop blocks and exact convolution formulas

G.1 When a product reduction is exact

Suppose the loop momenta partition into q disjoint sets $B_1, \ldots, B_q$ and every factor with a nonzero exponent depends on loop momenta from one set. External momenta may be shared. Equation (1) then has the form

$$I_{\boldsymbol{a}} = \int \prod_{b=1}^{q} d\mu_b \, f_b = \prod_{b=1}^{q} \int d\mu_b \, f_b = \prod_{b=1}^{q} I^{(b)}_{\boldsymbol{a}_b}. \tag{22}$$

Here $d\mu_b$ contains the normalized loop measures in block b, and $\boldsymbol{a}_b$ selects its active indices. The equality follows first in a convergence domain by iterated integration and then by dimensional analytic continuation, with the convention of Ref. [1]. A numerator that contains momenta from two proposed blocks prevents this factorization and joins those blocks in our construction. An unused auxiliary coordinate does not join them.

If block b reduces as $I^{(b)}_{\boldsymbol{a}_b} = \sum_j r^{(b)}_j M^{(b)}_j$, where $r^{(b)}_j$ are its coefficients and $M^{(b)}_j$ are its masters, substitution into Eq. (22) gives

$$I_{\boldsymbol{a}} = \prod_b \left(\sum_j r^{(b)}_j M^{(b)}_j \right). \tag{23}$$

This expression retains the complete reduction without distributing the product. Distribution would contain up to $\prod_b n_b$ terms if block b has n_b terms, since each expanded term chooses one entry from each factor. The factored expression stores $\sum_b n_b$ entries. These counts follow from the product construction; they do not claim that an arbitrary coupled integral can be factorized. Identical blocks with identical indices and kinematics require one reduction per request.

G.2 A massive tadpole

Let $T_a = \int d^dk/(i\pi^{d/2})(k^2 - m^2 + i0)^{-a}$. The total derivative of $k^\mu (k^2 - m^2)^{-a}$ gives

$$0 = dT_a - 2a(T_a + m^2 T_{a+1}), \qquad T_{a+1} = \frac{d - 2a}{2am^2} T_a. \tag{24}$$

We used $k^2 = (k^2 - m^2) + m^2$ in the first equality. Repeated substitution reduces each positive integer power to T_1. For $m^2 = 0$, the integral has no scale and vanishes in dimensional regularization [1]. This route makes no learned decision.

G.3 One massive line and massless vacuum lines

For $L \geq 2$, define $Q = \sum_{i=1}^{L} k_i$ and

$$V_{\boldsymbol{a},b} = \int \prod_{i=1}^{L} \frac{d^dk_i}{i\pi^{d/2}} \frac{1}{\prod_i (k_i^2 + i0)^{a_i} (Q^2 - m^2 + i0)^b}. \tag{25}$$

The scalar formula below assumes positive integer a_i, b, nonzero m^2, and no external momenta. It applies to this denominator structure at any L; our checks stop at thirteen loops.

To obtain its index dependence, work first in Euclidean signature. We use analytic continuation in d and the indices before evaluating the integer cases. Define $\Gamma(z) = \int_0^\infty u^{z-1} e^{-u} du$ for $\Re z > 0$, with analytic continuation elsewhere. The Schwinger identity and a Gaussian integral give

$$\begin{aligned} (k_E^2)^{-a} &= \frac{1}{\Gamma(a)} \int_0^\infty dt \, t^{a-1} e^{-tk_E^2}, \\ \int \frac{d^dk_E}{\pi^{d/2}} \frac{e^{2ik_E \cdot x}}{(k_E^2)^a} &= \frac{1}{\Gamma(a)} \int_0^\infty dt \, t^{a-1-d/2} e^{-x^2/t} = \frac{\Gamma(d/2 - a)}{\Gamma(a)} (x^2)^{a-d/2}. \end{aligned} \tag{26}$$

The final equality uses $u = x^2/t$ and the defining gamma integral. These parameter and continuation conventions agree with Ref. [1]. Put $A_0 = \sum_i a_i$ and $\alpha = A_0 - (L-1)d/2$. Multiplying the L transforms and inverting them gives the massless convolution proportional to

$$(Q_E^2)^{-\alpha} \frac{\Gamma(\alpha)}{\Gamma(d/2-\alpha)} \prod_i \frac{\Gamma(d/2-a_i)}{\Gamma(a_i)}. \tag{27}$$

The transform normalization is independent of the indices and cancels in the ratios used below. The remaining radial integral is

$$\begin{aligned} \int \frac{d^d Q_E}{\pi^{d/2}} \frac{(Q_E^2)^{-\alpha}}{(Q_E^2+m^2)^b} &= \frac{(m^2)^{d/2-\alpha-b}}{\Gamma(d/2)} \int_0^\infty \frac{u^{d/2-\alpha-1}}{(1+u)^b}\, du \\ &= (m^2)^{d/2-\alpha-b} \frac{\Gamma(d/2-\alpha)\Gamma(\alpha+b-d/2)}{\Gamma(d/2)\Gamma(b)}. \end{aligned} \tag{28}$$

The beta integral follows by $v = u/(1+u)$. Combining Eqs. (27) and (28) cancels $\Gamma(d/2-\alpha)$. Wick rotation supplies the relative sign $(-1)^{A_0+b-(L+1)}$ in our Minkowski denominator convention. Let $P = L+1$, $\Delta = A_0 + b - P$, and $(z)_n = \Gamma(z+n)/\Gamma(z)$. Dividing by the all-one integral gives

$$\frac{V_{\boldsymbol{a},b}}{V_{\mathbf{1},1}} = \frac{(-1)^\Delta (m^2)^{-\Delta} (P - Ld/2)_\Delta \, [L-(L-1)d/2]_{A_0-L}}{(b-1)! \prod_i [(a_i-1)!\,(d/2-a_i)_{a_i-1}]}. \tag{29}$$

Thus the production formula uses finite products, while the independent algebraic check uses the gamma ratios before expansion. If a shifted massless power is zero, its momentum can be integrated after shifting to Q, leaving a scaleless factor. If $b = 0$, the remaining massless integrals are also scaleless. These terms vanish under the stated assumptions.

G.4 One quadratic numerator

The tested extension allows one numerator v^2, where $v = \sum_i c_i k_i$ with fixed coefficients c_i. We write $V[v^2]$ for Eq. (25) with v^2 inserted in the numerator. Define $\beta_i = a_i - d/2$, $\lambda = \sum_i \beta_i$, and

$$C_v = \left(\sum_i c_i \beta_i \right)^2 - \sum_i c_i^2 \beta_i^2 = 2 \sum_{i<j} c_i c_j \beta_i \beta_j. \tag{30}$$

This equality is the expansion of the square. In the Fourier representation, a momentum insertion differentiates its corresponding factor in Eq. (26). The cross terms give $-C_v (x^2)^{\lambda-1}$, while multiplication of the full convolution by Q_E^2 gives

$$-\frac{1}{4} \partial_x^2 (x^2)^\lambda = -\lambda(\lambda + d/2 - 1)(x^2)^{\lambda-1}. \tag{31}$$

The derivative follows by applying the product rule twice. Dividing these two expressions, then returning to Minkowski signature, yields

$$V[v^2] = \sum_i c_i^2 V_{\boldsymbol{a}-\boldsymbol{e}_i,b} + \frac{C_v}{\lambda(\lambda+d/2-1)} \left(V_{\boldsymbol{a},b-1} + m^2 V_{\boldsymbol{a},b} \right). \tag{32}$$

The diagonal term uses k_i^2 to cancel one denominator power; the last term uses $Q^2 = (Q^2 - m^2) + m^2$. Equations (29) and (32) give rational coefficients in d and m^2. Apparent singularities require analytic limits. Higher numerator powers and other vacuum graphs remain outside this formula.

G.5 Massless two-point integrals with a quadratic numerator

Let $P = L + 1$, $q_i = k_i$ for $i \le L$, and $q_P = p - \sum_{i=1}^{L} k_i$. The massless two-point family is

$$W_{\boldsymbol{a}}(p) = \int \prod_{i=1}^{L} \frac{d^d k_i}{i\pi^{d/2}} \prod_{j=1}^{P} (q_j^2 + i0)^{-a_j}, \qquad p^2 \neq 0. \tag{33}$$

Here all a_j are positive integers. Applying Eq. (26) to the P massless factors and inverting their product gives the convolution coefficient

$$\mathcal{C}(\boldsymbol{a}) = \frac{\Gamma(A - Ld/2)}{\Gamma(Pd/2 - A)} \prod_{j=1}^{P} \frac{\Gamma(d/2 - a_j)}{\Gamma(a_j)}, \qquad A = \sum_j a_j. \tag{34}$$

The momentum power is $Ld/2 - A$, as follows either from the inverse transform or from mass dimension. Dividing by the all-one result and cancelling the relative Wick-rotation signs for integer $A - P$ gives

$$\frac{W_{\boldsymbol{a}}}{W_{\boldsymbol{1}}} = (p^2)^{P-A} \frac{\mathcal{C}(\boldsymbol{a})}{\mathcal{C}(\boldsymbol{1})}. \tag{35}$$

The gamma shift identity turns this ratio into the finite rising products used by the scalar implementation. The derivation uses the same dimensional continuation as the preceding vacuum calculation.

Write $W[v^2]$ for Eq. (33) with numerator v^2. For $v = \sum_{i=1}^{L} c_i k_i + c_p p$, use $p = \sum_{j=1}^{P} q_j$ to write $v = \sum_{j=1}^{P} w_j q_j$, where $w_i = c_i + c_p$ for $i \le L$ and $w_P = c_p$. Define $\beta_j = a_j - d/2$, $\lambda = \sum_j \beta_j$, and $C_w = (\sum_j w_j \beta_j)^2 - \sum_j w_j^2 \beta_j^2$. The derivative calculation in Eqs. (30) and (31) now applies to all P factors. Since their total momentum is p, it gives

$$W[v^2] = \sum_{j=1}^{P} w_j^2 W_{\boldsymbol{a}-\boldsymbol{e}_j} + \frac{C_w p^2}{\lambda(\lambda + d/2 - 1)} W_{\boldsymbol{a}}. \tag{36}$$

A term with a removed massless line has a scaleless integration and vanishes. Equations (35) and (36) reduce the supported scalar and single-numerator targets to $W_{\boldsymbol{1}}$. The formula allows an external momentum in the numerator and excludes multiple numerator factors, nonzero internal masses, and more general two-point graphs.

H Derivation of the complete-time training rule

For a target batch i and an initial Kira setting a, let T_{ia} be the measured time from application launch to exact comparison and process completion. This includes loading, selection, equation generation, algebra, and every recovery attempt. We define the training cost

$$\widetilde{T}_{ia}^{(q)} = \begin{cases} T_{ia}^{(q)}, & \text{verified}, \\ \max(220\,\mathrm{s}, T_{ia}^{(q)}), & \text{unverified}, \end{cases} \qquad C_{ia} = \operatorname{median}_q \widetilde{T}_{ia}^{(q)}. \tag{37}$$

Here q labels repetitions. The 220-second penalty is a training choice that makes an unfinished calculation costly; it does not estimate its unknown completion time. The results use measured seconds and completion counts, with no penalty substituted into reported speedups.

We construct a tree from structural features of the target integrals and their required master basis. In a leaf containing training requests S, choosing one action a incurs total cost $\sum_{i \in S} C_{ia}$. Minimizing this expression gives the leaf action and cost,

$$a_S = \arg\min_a \sum_{i \in S} C_{ia}, \qquad J(S) = \min_a \sum_{i \in S} C_{ia}. \tag{38}$$

A candidate feature threshold divides S into disjoint sets S_- and S_+. The two leaves can choose their actions separately, so their minimum combined cost is $J(S_-) + J(S_+)$. We therefore choose the threshold that gives the largest positive decrease

$$\Delta J = J(S) - J(S_-) - J(S_+). \tag{39}$$

This derivation trains the partition directly from elapsed reduction cost. Cost-sensitive algorithm selection also uses unequal penalties for expensive choices [16]. Our particular split rule follows from Eqs. (38)–(39); it does not certify a globally optimal tree or reduction path.

Each leaf stores the mean cost for every candidate action. A single tree predicts these means, while a forest averages the means from its reached leaves. If $\widehat{C}_a$ is the predicted cost, a_0 is the validation-selected anchor, δ is a decision margin, and $\tau = 0.002\,\mathrm{s}$ is the decision allowance, the candidate $a_* = \arg\min_a \widehat{C}_a$ replaces the anchor only when

$$\widehat{C}_{a_*} + \tau < (1-\delta)\widehat{C}_{a_0}. \tag{40}$$

The condition requires the predicted saving to exceed the allowance and margin. Validation must also show at least 1.5% less cost than the best fixed candidate, with no increase in unverified requests, before the application enables learned decisions. Inputs outside the recorded structural ranges use native settings. These tests control deployment; they do not guarantee a saving on a new input.

The master-basis check also informs the seed region. For an integral with indices $\boldsymbol{a}$, define $r(\boldsymbol{a}) = \sum_j \max(a_j, 0)$, $s(\boldsymbol{a}) = \sum_j \max(-a_j, 0)$, $u(\boldsymbol{a}) = \sum_j \max(a_j - 1, 0)$, and $t(\boldsymbol{a}) = \sum_j \mathbf{1}_{a_j>0}$. These are the positive-power sum, numerator rank, number of dots, and number of positive indices. For the six basis-compatible candidates, we enlarge the corresponding bounds to contain the supplied basis. The unadjusted native candidate retains its original bounds. Kira's additional subsector restriction is $s \le \max(1, t-\ell+1, s_{\mathrm{set}})$, where s_{set} is the explicitly assigned sector limit [5]. Our first selector uses the stronger sufficient condition

$$\ell \le \min_M \{t(M) - s(M)\}. \tag{41}$$

We lower a proposed cutoff when needed to satisfy Eq. (41). It gives $t(M) - \ell + 1 \ge s(M) + 1$, so each master passes the extra cutoff once the outer bounds contain it. This conservative choice can admit more seeds than necessary. It does not prove that the resulting identities suffice, so the application retains the exact master-basis check and less restrictive recovery settings.

H.1 Target batches and basis features

The denominator definitions and rational kinematic profiles follow Appendix D. We specify the new target powers here so that the training and test batches have an explicit mathematical definition. Let n be the number of physical denominators, let N include the auxiliary coordinates, and define $b_j = 1$ for $j \le n$ and $b_j = 0$ otherwise. For profile integer h and target number $q = 0, \ldots, \min(n, 3) - 1$, set

$$i_q = 1 + ((h+q) \bmod n), \qquad k_q = n + 1 + ((h+q) \bmod (N-n)), \qquad v_q = 1 + (i_q \bmod n). \tag{42}$$

The index k_q is used only when $N > n$. With $\boldsymbol{e}_j$ denoting unit vector j, the four target shapes are

$$\boldsymbol{a}_q^{\mathrm{dots}} = \boldsymbol{b} + (1 + \mathbf{1}_{q=1})\boldsymbol{e}_{i_q}, \tag{43}$$

$$\boldsymbol{a}_q^{\mathrm{mixed}} = \boldsymbol{b} + \boldsymbol{e}_{i_q} - \boldsymbol{e}_{k_q}, \tag{44}$$

$$\boldsymbol{a}_q^{\mathrm{rank2}} = \boldsymbol{b} - 2\boldsymbol{e}_{k_q}, \tag{45}$$

$$\boldsymbol{a}_q^{\mathrm{pinched}} = \boldsymbol{b} - \boldsymbol{e}_{i_q} + \boldsymbol{e}_{v_q}. \tag{46}$$

Here $\mathbf{1}_{q=1}$ equals one for $q = 1$ and zero otherwise. If there is no auxiliary coordinate, the mixed shape omits its numerator term and the rank-two shape becomes $\boldsymbol{b} + 2\boldsymbol{e}_{i_q}$. The pinched shape removes one physical denominator and increases the next power by one. Training uses the first three shapes, validation uses the first two, and testing uses all four. We remove requests already handled by the exact formulas when defining the native training and validation collection. The pentagon structure is also excluded from both sets.

Independent products use the dotted shape for each vacuum, massive two-point, or ladder block. Their nonplanar box blocks and one-loop box remainders instead use $\boldsymbol{b} + 2\boldsymbol{e}_{i_q}$ for each target. Blocks share external invariants and have disjoint loop momenta; cross-block auxiliary indices remain zero. At even loop order, the separate analytic test uses $\boldsymbol{b} + \boldsymbol{e}_{i_q} - \boldsymbol{e}_{k_q}$ when an auxiliary coordinate exists, and omits the numerator otherwise. At odd loop order it uses $\boldsymbol{b} + 2\boldsymbol{e}_{i_q}$. The notes give the disjoint profile integers and preserve every resulting index vector in the measurement records.

For a nonempty master basis $\mathcal{B}$, define $r_B = \max_{M\in\mathcal{B}} r(M)$, $s_B = \max_{M\in\mathcal{B}} s(M)$, $u_B = \max_{M\in\mathcal{B}} u(M)$, and $\ell_B = \min_{M\in\mathcal{B}}[t(M) - s(M)]$. Let s_T and u_T be the corresponding target maxima, and write $x_+ = \max(x, 0)$. The seven added features are

$$\boldsymbol{z}_B = (|\mathcal{B}|, r_B, s_B, u_B, \ell_B, (s_B - s_T)_+, (u_B - u_T)_+). \tag{47}$$

These quantities describe the required basis before reduction. They contain no coefficient values. The implementation uses zeros for the basis summaries when the list is empty and applies no basis-dependent enlargement in that case.

H.2 The seven candidate settings

Let r_T, s_T, u_T be target maxima and let t_{top} be the largest positive-index count among the declared top sectors. The unadjusted native candidate uses $(\max(t_{\text{top}}, r_T), s_T, u_T)$ and no subsector truncation. For an adjusted candidate with offsets $(\Delta r, \Delta s, \Delta u)$, the bounds are

$$(r, s, u) = (\max(t_{\text{top}}, r_T + \Delta r, r_B), \max(s_T + \Delta s, s_B), \max(u_T + \Delta u, u_B)). \tag{48}$$

Table 21 completes the action definition. A dash disables the extra subsector cutoff. The native region, raised rank, and expanded region provide recovery choices when a restricted region fails its exact basis check.

Table 21: The six basis-compatible candidates, in addition to the unadjusted native candidate.

Candidate	$(\Delta r, \Delta s, \Delta u)$	Subsector cutoff ℓ
Compatible native	$(0, 0, 0)$	–
Increased rank	$(0, 1, 0)$	–
Enlarged region	$(1, 1, 1)$	–
Truncated	$(0, 0, 0)$	$\min(t_{\text{top}} - s_T, \ell_B)$
Truncated with increased rank	$(0, 1, 0)$	$\min(t_{\text{top}} - s_T, \ell_B)$
Relaxed truncation	$(0, 0, 0)$	$\min(t_{\text{top}} - s_T - 1, \ell_B)$

I A tighter condition for the master-basis cutoff

The cost objective, target definitions, and 51 inputs remain those in Appendix H. The new model learns from fresh profiles, and all nine candidate costs for one request are measured together on the same worker. The notes give the training and test profiles and the selected model parameters.

Kira restricts the numerator rank in a sector through [5]

$$s \le \max\{1, t - \ell + 1, s_{\text{set}}\}. \tag{49}$$

Here t is the sector's positive-index count, ℓ is the truncation parameter, and s_{set} is the rank limit explicitly assigned to that sector. A master with rank $s_M \le \max(1, s_{\text{set},M})$ already satisfies this additional cutoff for any ℓ. For every other master, Eq. (49) reduces to $s_M \le t_M - \ell + 1$. Rearranging gives $\ell \le t_M + 1 - s_M$.

Let $\mathcal{H}$ contain precisely the masters with $s_M > \max(1, s_{\text{set},M})$. If $\ell_0 = t_{\text{top}} - s_T$ is the proposed truncation value, the largest value no greater than ℓ_0 that admits all required masters is therefore

$$\ell_* = \min\left(\ell_0, \min_{M \in \mathcal{H}} \{t_M + 1 - s_M\}\right). \tag{50}$$

An empty inner set imposes no bound, so $\ell_* = \ell_0$ in that case. Taking the minimum enforces each master's inequality, while any larger value would violate at least one of the active bounds or exceed the proposed value. Thus Eq. (50) is the least enlargement needed for this particular cutoff-admission condition. It does not determine a sufficient IBP system or a minimum reduction time.

The two added candidates use ℓ_* with the ordinary or increased numerator-rank bound. Their outer positive-power, numerator-rank, and dot bounds still contain the supplied master basis. The explicitly seeded top sectors use the candidate's assigned rank bound; other sectors have $s_{\text{set}} = 0$ unless explicitly declared. The previous seven candidates remain available, so the trained policy can select a conservative setting when its complete cost is lower. Every candidate keeps the exact coefficient and master-basis checks, and failed attempts retain their recovery cost in training and testing.

J Definition of the hybrid expert comparison

The sparse expert uses the nonplanar denominators in Eq. (16). Its targets have seven unit physical powers. For numerator rank $R = 2, 3, 4$, we include

$$\boldsymbol{a}_R^{(1)} = (1, 1, 1, 1, 1, 1, 1, -R, 0), \qquad \boldsymbol{a}_R^{(2)} = (1, 1, 1, 1, 1, 1, 1, 0, -R). \tag{51}$$

For $R = 3, 4$, we also include

$$\boldsymbol{a}_R^{(12)} = (1, 1, 1, 1, 1, 1, 1, -\lceil R/2 \rceil, -\lfloor R/2 \rfloor). \tag{52}$$

These definitions give eight training and validation targets per kinematic profile. The reserved collection additionally includes every vector $(1, 1, 1, 1, 1, 1, 1, -j, -(R - j))$ for $j = 0, \ldots, R$ and $R = 2, 3, 4$, giving twelve targets per profile. We use $s = -113 - 18h$ and $t = -127 - 10h$, with $h = 2301, 2303$ for training, 2309 for validation, and $2311, 2317$ for reserved tests. The additional two outer-model inputs are $\max(0, -a_8)$ and $\max(0, -a_9)$, appended to the 51 inputs in Appendix H. Their coordinate meanings remain fixed within this ordered family.

The outer actions are the preceding basis-compatible setting, the expert's fixed sparse boundary, its learned boundary, and original native bounds. The best ML-free control minimizes validation cost over all but the learned boundary. The cost-sensitive tree objective follows Eqs. (37)–(40), with complete costs for these four actions. The outer prediction is retained only if it improves on the best constant action by at least 1.5%. The resulting application must also improve on the best ML-free candidate by at least 1.5%, with no increase in unverified requests. A constant choice of the learned expert still performs that expert's target-dependent boundary prediction.

For the inner expert, the boundary coefficients $\theta_0, \ldots, \theta_6$ have the definitions in Appendix B. The saved forest predicts log-time ratios for these candidates. It changes the fixed candidate

only when the predicted decrease exceeds its frozen margin. Table 22 records the candidate coefficients; the native row applies no sparse boundary. Target and master seeds that satisfy the outer native bounds remain explicitly included. The native master-basis and coefficient checks determine acceptance, and a failed sparse attempt returns to less restrictive native work.

Table 22: Frozen sparse-boundary candidates used by the inner expert. Coefficients define the seed region of Eq. (8).

Candidate	θ_0	θ_1	θ_2	θ_3	θ_4	θ_5	θ_6
0: native	–	–	–	–	–	–	–
1	1	0	0	1	0	1	0
2	2	0	0	1	0	1	0
3	1	0	0	0	0	1	0
4	1	0	0	0	1	1	0
5	1	0	0	-1	0	1	0
6	1	0	0	1	1.5	1	0
7	1.5	-0.25	0	1.25	0	1	-0.25
8	2	0	-0.25	1	0	1	0
9	2.5	0	0	1.25	0	1	-0.25

The fixed boundary index is 4 and the inner decision margin is 0. The ten compatibility requests use profile 2321 for the three original coupled native families, a pentagon, a rank-three massive two-point target, and the one-mass analytic family, and profiles $2327 + 4L$ for four independent products. Their denominators and target rules remain those in Appendices D and H.1. They provide checks at loop orders one, two, three, four, eight, twelve, and thirteen.

For the selected tree, let $u = \max(0, -a_8)$ be the first auxiliary numerator power and let $R = \max(0, -a_8) + \max(0, -a_9)$ be the total numerator rank. The first split tests $u \leq 2.5$, and its lower branch tests $R \leq 3.5$. Since both quantities are integers, the deployed choice within the expert domain is

$$\pi(u, R) = \begin{cases} \text{learned sparse boundary}, & u \leq 2,\ R \leq 3, \\ \text{fixed sparse boundary}, & u \leq 2,\ R = 4, \\ \text{preceding basis-compatible setting}, & u \geq 3. \end{cases} \tag{53}$$

Here $\pi(u, R)$ is the selected reduction route. Each leaf selects the route with its lowest predicted complete cost, subject to the decision allowance of Eq. (40). For the first two leaves, the predicted saving relative to the anchor exceeds the 0.002-second allowance. The third leaf already selects the anchor, so the allowance leaves its choice unchanged. These comparisons give Eq. (53). Thus the outer tree can select a fixed or native setting even within the sparse expert's supported family. The exact checks and recovery rules still apply to each selected route.

K An exact recurrence for scalar one-loop integrals

This appendix derives the recurrence used by the analytic stage. It follows from the same IBP identity as Eq. (2); analytic one-loop reduction and its matrix formulation also appear in Ref. [8]. Our purpose is to apply the identity before constructing a larger equation system.

Write the denominators of a one-loop family as

$$D_i = (\ell + Q_i)^2 - m_i^2 + i0, \qquad Y_{ij} = m_i^2 + m_j^2 - (Q_i - Q_j)^2. \tag{54}$$

The vectors Q_i contain external momenta only. The symmetric matrix Y depends on the masses and external invariants, and it equals the negative of the modified Cayley matrix convention used

in Ref. [8]. A sector S contains the indices whose powers are positive. We apply the following derivation to scalar targets, so the remaining powers vanish.

For a positive seed vector $\boldsymbol{b}$ on S, choose $v = \ell + Q_i$ in the IBP identity. The elementary relation

$$2(\ell + Q_i) \cdot (\ell + Q_j) = D_i + D_j + Y_{ij} \tag{55}$$

expresses every numerator produced by the derivative through the denominators. The infinitesimal pole prescription does not change these algebraic identities. Differentiation gives

$$0 = dI_{\boldsymbol{b}} - \sum_{j\in S} b_j \int \frac{d^d\ell}{i\pi^{d/2}} \frac{D_i + D_j + Y_{ij}}{D_j} \prod_{k\in S} D_k^{-b_k}, \tag{56}$$

$$\sum_{j\in S} Y_{ij} b_j I_{\boldsymbol{b}+\boldsymbol{e}_j} = (d-B) I_{\boldsymbol{b}} - \sum_{k\in S} b_k I_{\boldsymbol{b}+\boldsymbol{e}_k-\boldsymbol{e}_i}, \qquad B = \sum_{k\in S} b_k. \tag{57}$$

Here $\boldsymbol{e}_j$ has entry one at position j and zeros elsewhere. The matrix of this system has one row and one column for each active denominator. If its sector submatrix Y_S is invertible, the raised integral is therefore

$$I_{\boldsymbol{b}+\boldsymbol{e}_j} = \frac{1}{b_j} \sum_{i\in S} (Y_S^{-1})_{ji} \left[(d-B) I_{\boldsymbol{b}} - \sum_{k\in S} b_k I_{\boldsymbol{b}+\boldsymbol{e}_k-\boldsymbol{e}_i} \right]. \tag{58}$$

To reduce a target $\boldsymbol{a}$ with $a_j > 1$, set $\boldsymbol{b} = \boldsymbol{a} - \boldsymbol{e}_j$. Every integral on the right has total positive power $\sum_i a_i - 1$, and none has a negative power. Repetition therefore terminates at unit-power sectors or scaleless integrals, which vanish in dimensional regularization [1]. The implementation shares repeated intermediate integrals within the request and inverts each required sector matrix once.

This derivation also determines the coefficient arithmetic. For rational masses and invariants, Y_S^{-1} has rational entries, while the factor $d - B$ is linear in d. Induction on the total positive power shows that each coefficient in the unit-power decomposition is a polynomial in d with rational coefficients. For symbolic kinematics $\boldsymbol{x}$, the same argument gives polynomials in d over $\mathbb{Q}(\boldsymbol{x})$. Thus exact polynomial arithmetic suffices for this recurrence; no numerical approximation of a coefficient is required.

Unit-power sectors can obey further identities at special kinematics. The application accepts the analytic result only when every remaining integral belongs to its independently prepared Kira basis, allowing the exact equality of translated tadpoles with the same mass. For such a tadpole, the substitution $\ell' = \ell + Q_i$ has unit Jacobian and removes the external shift from its single denominator. A missing basis element, a required singular matrix, an unsupported numerator, or an exceeded analytic work limit sends the request to the existing Kira application. The existing application interface rejects cut propagators and symbolic indices before this stage. This check preserves a common output basis for the timing comparisons. It also states the scope of the analytic method: Eq. (58) reduces supported scalar one-loop sectors and independent products containing them, while connected multiloop blocks continue through the other exact or learned methods.

L Inputs and comparison protocol for the analytic stage

For a polygon with n denominators, use one loop momentum and $n-1$ independent external vectors, with $Q_1 = 0$ and $Q_j = -\sum_{i=1}^{j-1} p_i$. At integer profile h, assign

$$m_j^2 = 2 + 3(j-1) + (h \bmod 17), \tag{59}$$

$$p_i^2 = -[11 + (h \bmod 23) + 7(i-1)], \tag{60}$$

$$p_i \cdot p_j = -[1 + ((h+i+j-2) \bmod 3)], \quad i < j. \tag{61}$$

These equations define the test inputs rather than physical identities. Use $h = 2411, 2417$, $n = 2, 3, 4, 5$, and extra power $\delta = 1, 3$. Each request raises one of the first $\min(3, n)$ denominators by δ and also includes a target with one extra power on each endpoint denominator. Duplicate targets are removed. The two symbolic requests use $n = 2$ and keep $\mu_1 = m_1^2$, $\mu_2 = m_2^2$, and $s = p_1^2$ as independent variables, with extra powers one and two.

For each loop count $L = 2, \ldots, 13$, form a product of L independent copies of the two-point family at profile $2441 + L$, with extra power $1 + (L \bmod 3)$. The copies share external kinematics, and the application keeps their master expansion factored. The eight coupled nonplanar targets use the definitions in Appendix J, profile 2473, and the original eight numerator patterns. The remaining profile-2477 requests are the massive two-loop propagator, all-mass three-loop vacuum, and one-mass thirteen-loop vacuum families defined in Appendix D.

Before timing, native Kira reduces each new target collection together with all nonempty unit-power sectors. A second native calculation changes integral ordering and seed bounds, and exact coefficient agreement defines the reference. Only the resulting master indices enter the application. All measured methods use the same reference for the final coefficient check. The analytic stage accepts at most six extra powers and five denominators for rational kinematics, or three extra powers and three denominators for symbolic kinematics. Its bounded attempt returns unsupported cases to the existing application, with all elapsed work retained in the timer.

M Inputs and admission rule for the final application

Use the symbolic two-point denominators $D_1 = \ell^2 - \mu_1 + i0$ and $D_2 = (\ell - p)^2 - \mu_2 + i0$, with independent variables μ_1, μ_2, and $s = p^2$. Development uses one target per request, with powers $(4, 1)$ or $(1, 4)$. Testing uses $(3, 2)$ and $(2, 3)$, one product of two identical $(3, 2)$ blocks, and one product of thirteen identical $(2, 3)$ blocks. As in the main comparison, identical blocks share work within each request and the product expansion stays factored.

The compatibility inputs comprise two-, four-, and five-denominator rational polygons at profile 2521 and extra power three, using Appendix L. They also include a nonplanar box at profile 2527 with target $(1, 1, 1, 1, 1, 1, 1, -3, 0)$ and the one-mass thirteen-loop vacuum at profile 2531, with two extra powers on its first physical denominator. Their family definitions follow Appendices J and D. Every input uses the previously prepared master indices; the test does not add entries to that registry.

The input rule accepts direct integer or rational mass and scalar-product values for the new recurrence. Expressions that require a symbolic parser use the existing exact or native path, even if an expression could simplify to a rational number. This conservative implementation avoids a symbolic startup cost merely to decide which method to use. Three rotated repetitions compare the final learned application, its fixed counterpart, matched native settings, and the initial analytic-enabled application. Exact comparison with an independent reference follows every run and remains inside its timer.

N Combined recurrence and learned strategy

For the target $\boldsymbol{a} = \boldsymbol{b} + \boldsymbol{e}_j$, define

$$w_i = \frac{(Y_S^{-1})_{ji}}{b_j}, \qquad W = \sum_{i \in S} w_i. \tag{62}$$

Here S, Y_S, and $\boldsymbol{b}$ have the definitions in Appendix K. In the double sum of Eq. (58), the term with $k = i$ has index vector $\boldsymbol{b} + \boldsymbol{e}_i - \boldsymbol{e}_i = \boldsymbol{b}$. Separating these terms gives

$$I_{\boldsymbol{a}} = (d - B)W I_{\boldsymbol{b}} - \sum_{i \in S} w_i b_i I_{\boldsymbol{b}} - \sum_{\substack{i,k \in S \\ k \neq i}} w_i b_k I_{\boldsymbol{b}+\boldsymbol{e}_k-\boldsymbol{e}_i} \tag{63}$$

$$= \left[(d - B)W - \sum_{i \in S} w_i b_i\right] I_{\boldsymbol{b}} - \sum_{\substack{i,k \in S \\ k \neq i}} w_i b_k I_{\boldsymbol{b}+\boldsymbol{e}_k-\boldsymbol{e}_i}. \tag{64}$$

This groups the coefficient of the same integral before further substitution. Its algebra, termination, and allowed master check are unchanged from the earlier recurrence. For rational kinematics, each w_i is rational, so exact integer numerator and denominator arithmetic suffices. The dimension remains symbolic in the polynomial coefficient.

The direct scalar formulas use rising factorials. For $\Re z > 0$, integration by parts in the defining gamma integral gives

$$\Gamma(z + 1) = [-t^z e^{-t}]_0^\infty + z \int_0^\infty t^{z-1} e^{-t}\, dt = z\Gamma(z). \tag{65}$$

Here t is the positive integration variable and z is the complex argument. Applying this identity repeatedly gives

$$(z)_n = \frac{\Gamma(z + n)}{\Gamma(z)} = \prod_{r=0}^{n-1} (z + r), \qquad (z)_0 = 1, \tag{66}$$

for a nonnegative integer n. The equality extends by analytic continuation away from unresolved poles. Thus the ratios derived in Appendix G can be evaluated through finite products without constructing gamma functions. The references use the earlier symbolic implementation, including gamma-function ratios for the multiloop vacuum and numerator requests. Scalar massless requests use its rising-factorial expression. The polygon recurrence uses independent native Kira reductions. This change concerns evaluation cost; it does not extend the mathematical domain of those formulas.

The scaleless-vacuum shortcut follows directly from IBP in dimensional regularization [2, 1]. With no masses or external momenta, every inverse propagator is homogeneous of degree two in the loop momenta. If $A = \sum_j a_j$, the product of propagator powers has degree $-2A$. Summing the IBP operators with $v = \ell_h$ therefore gives

$$0 = \int \prod_h \frac{d^d \ell_h}{i\pi^{d/2}} \sum_{h=1}^{L} \frac{\partial}{\partial \ell_h^\mu} \left(\ell_h^\mu \prod_j D_j^{-a_j} \right) = (Ld - 2A) I_{\boldsymbol{a}}. \tag{67}$$

The factor $Ld - 2A$ is nonzero at generic symbolic d, so the dimensionally regularized integral vanishes. Analytic continuation supplies the same scaleless result. This argument also permits negative integer powers because they preserve homogeneity. The six additional audit completions at eight through thirteen loops use this zero identity; they do not require a native system above its index limit.

For a target collection $\mathcal{T}$ with q targets and n denominator positions, define the extra positive power of target $\boldsymbol{a}$ by $\delta(\boldsymbol{a}) = \sum_{j=1}^n \max(a_j - 1, 0)$. The model inputs are

$$\boldsymbol{f} = \left(n, q, \max_{\mathcal{T}} \delta, q^{-1} \sum_{\mathcal{T}} \delta, \max_{\boldsymbol{a} \in \mathcal{T}, j} a_j, \min_{\boldsymbol{a} \in \mathcal{T}} \sum_j \mathbf{1}_{a_j > 0}, \max_{\mathcal{T}} a_1, \ldots, \max_{\mathcal{T}} a_5 \right). \tag{68}$$

Absent positions have value zero. These are definitions of the representation, and $\mathbf{1}$ is the indicator function. Coordinate positions enter this small one-loop model; we do not claim invariance under arbitrary relabeling.

Let t_{rc} be the median measured recurrence time for training request r and strategy c. A leaf containing requests R stores the four mean costs $|R|^{-1}\sum_{r\in R} t_{rc}$. We grow an axis-aligned regression tree using the tree framework of Ref. [10], with a cost-sensitive split criterion. For a proposed partition into R_1 and R_2, the minimized empirical action cost is

$$J(R_1,R_2)=\sum_{u=1}^{2}\min_{c}\sum_{r\in R_u} t_{rc}. \tag{69}$$

The equation follows by assigning one strategy to each leaf and summing its recorded costs. We accept a split only when it lowers this sum by more than $10^{-4}|R|$ seconds and satisfies the depth and population limits, where $|R|$ is the number of requests in the unsplit node. Candidate depths are one through five, and minimum leaf populations are three, six, or ten. Validation compares prediction margins 0, 0.03, and 0.10 against the best single strategy. The selected model requires a two-percent aggregate validation saving and at least one changed decision before deployment.

For predicted costs $\widehat{t}_c$ and fixed strategy c_0, choose $c_*\in\arg\min_c \widehat{t}_c$. The application uses that change only when

$$\widehat{t}_{c_*}+\tau<(1-\eta)\widehat{t}_{c_0},\qquad \tau=10^{-4}\ \mathrm{s}, \tag{70}$$

where η is the selected margin and τ allows for prediction work. Inputs outside the observed feature ranges use the fixed strategy. This rule is a design choice rather than a proof of a runtime improvement. Complete reserved timings measure its actual effect.

O Inputs for the extended comparison

Training uses polygon kinematics at profiles 2611 and 2617, and validation uses profile 2621. For each number of denominators $n=2,3,4,5$, each profile supplies fourteen target collections. Starting from unit powers, the generator distributes one through six extra powers among the denominator positions. Every third collection contains two targets, and the others contain one. The reserved profiles and target collections below are separate.

The timing set contains 65 requests. At each loop count $L=1,\dots,13$, it includes a one-mass vacuum banana and a massless propagator banana from Appendix D, with profile $2701+2L$. It also includes a product of L independent massive triangles. Each triangle uses the polygon kinematics of Appendix L at profile 2741, adds $7b+L$ to each squared mass for block $b=0,\dots,L-1$, and has target powers $(2,1,1)$. These distinct masses require separate block reductions within a request.

Four further one-mass vacuum requests keep their scale symbolic at $L=1,2,6,13$. Nine scalar recurrence requests use polygons with $n=3,4,5$ at profile 2777 and powers $(3,2,1,\dots,1)$, $(1,\dots,1,2,3)$, or $(2,\dots,2)$. Two fully symbolic bubbles use profile 2783 with the two power collections defined in Appendix L. Eight nonplanar-box requests at profile 2791 use unit physical powers and the auxiliary pairs $(-2,0)$, $(0,-2)$, $(-3,0)$, $(0,-3)$, $(-2,-1)$, $(-4,0)$, $(0,-4)$, and $(-2,-2)$. Three compatibility requests use a massive two-loop propagator, an all-mass three-loop vacuum, and a two-loop ladder at profile 2797, with the first physical power raised by one.

A separate 110-request audit tests the six vacuum, propagator, and scattering sequences of Appendix D at each loop count through thirteen, excluding two duplicate one-loop definitions. It adds thirteen scaleless vacuum requests, six scattering requests with mixed or pinched powers, and fifteen symbolic-scale requests. Both the preceding and new application receive a twenty-second limit for each audit request. The audit records exact agreement when both complete, as well as time limits, missing basis support, and native input limits. These bounded outcomes do not prove that a failed integral has no reduction.

The native solver used here has a documented maximum of 31 integral indices [5]. A complete independent scalar-product coordinate system has $L(L+1)/2+LE$ entries for E external

momenta, from the $L(L+1)/2$ symmetric loop–loop products and LE loop–external products. Analytic formulas and independent blocks can handle some requests above this native bound. A coupled request outside both the exact domain and the native limit remains unsupported. The finite audit also leaves arbitrary numerators, singular kinematics, cut propagators, and symbolic propagator powers outside a universal coverage claim.

P Inputs for the complete-cost deployment test

Use the polygon definitions of Appendix L with profile 2917, $n = 3, 4, 5$, and powers $(3, 2, 1, \ldots, 1)$ or $(1, \ldots, 1, 2, 3)$. The distinct-triangle products contain two or thirteen blocks at profile 2927, with the mass offsets and target $(2, 1, 1)$ defined in Appendix O. The nonplanar box has profile 2939 and auxiliary powers $(-3, 0)$. The symbolic bubble has profile 2941 and the first extra-power collection of Appendix L. The one-mass vacuum and massless propagator families have thirteen loops and profile 2953.

The full-cost admission uses only the original profile-2621 validation inputs. Its cost includes a fresh process, the learned or fixed choice, exact recurrence, output, and comparison with the earlier exact scalar recurrence. The additional reserved references use the same independent native and analytic checks as the main test. We fix all source choices and the admission result before these twelve requests run. The final rule changes no fitted weights. It selects whether the recurrence model earns its cost in the complete application.

Q Exact one-scale triangles and shared target reductions

The new exact rule applies to a massless triangle whose two adjacent external momenta satisfy $p_1^2 = p_2^2 = 0$, while $s = (p_1 + p_2)^2 \neq 0$. We use the same Minkowski denominator convention as the preceding integral definitions and write

$$T(a,b,c) = \int \frac{\mathrm{d}^d \ell}{i\pi^{d/2}} \frac{1}{[\ell^2 + i0]^a [(\ell - p_1)^2 + i0]^b [(\ell - p_1 - p_2)^2 + i0]^c}, \qquad \mathcal{B}(s,d) = T(1,0,1). \tag{71}$$

Here a, b, c are integer propagator powers and $\mathcal{B}(s,d)$ is the off-shell bubble master integral. The rule keeps d symbolic. It does not evaluate the master integral numerically.

Substituting the three external shifts and zero internal masses into Eq. (54) gives

$$Y = \begin{pmatrix} 0 & 0 & -s \\ 0 & 0 & 0 \\ -s & 0 & 0 \end{pmatrix}, \qquad \det Y = 0. \tag{72}$$

The zero middle row makes the full triangle matrix singular. The preceding recurrence requires its inverse, so this sector needs another identity. The parameter integral below provides that identity directly.

Let $A = a + b + c$. Feynman parametrization and the loop-momentum integral give the standard simplex representation [1]. For this triangle the quadratic form is $-sx_1x_3 - i0$, so the representation becomes

$$T(a,b,c) = \frac{(-1)^A \Gamma(A - d/2)}{\Gamma(a)\Gamma(b)\Gamma(c)} (-s - i0)^{d/2 - A} \int_{x_j \geq 0} \mathrm{d}x_1\, \mathrm{d}x_2\, \mathrm{d}x_3\, \delta(1 - x_1 - x_2 - x_3) \times x_1^{d/2-b-c-1} x_2^{b-1} x_3^{d/2-a-b-1}. \tag{73}$$

We first take powers and dimension in a convergence domain and then continue the result meromorphically. Here Γ is Euler's gamma function. Its integral definition gives a useful relation for

the parameter integrals:

$$\begin{aligned}\Gamma(\alpha)\Gamma(\beta) &= \int_0^\infty \mathrm{d}t \int_0^\infty \mathrm{d}r\, t^{\alpha-1} r^{\beta-1} e^{-t-r} \\ &= \Gamma(\alpha+\beta) \int_0^1 \mathrm{d}z\, z^{\alpha-1}(1-z)^{\beta-1}. \end{aligned} \tag{74}$$

The second line follows from $\rho = t + r$, $z = t/(t + r)$, and the Jacobian ρ. The ρ integral equals $\Gamma(\alpha + \beta)$, which derives the beta integral in terms of gamma functions. To evaluate the simplex integral, set $x_1 = u$, $x_2 = (1 - u)v$, and $x_3 = (1 - u)(1 - v)$. The Jacobian is $1 - u$, and two applications of Eq. (74) give

$$\int \mathrm{d}^3x\, \delta(1 - \textstyle\sum_j x_j) \prod_{j=1}^3 x_j^{\alpha_j - 1} = \frac{\Gamma(\alpha_1)\Gamma(\alpha_2)\Gamma(\alpha_3)}{\Gamma(\alpha_1+\alpha_2+\alpha_3)}. \tag{75}$$

The three exponents in Eq. (73) correspond to $\alpha_1 = d/2 - b - c$, $\alpha_2 = b$, and $\alpha_3 = d/2 - a - b$. Substitution cancels $\Gamma(b)$ and yields

$$T(a, b, c) = (-1)^A(-s - i0)^{d/2-A} \frac{\Gamma(A - d/2)\Gamma(d/2 - a - b)\Gamma(d/2 - b - c)}{\Gamma(a)\Gamma(c)\Gamma(d - A)}. \tag{76}$$

The same continued expression supplies $b = 0$ and negative integer b, where the middle inverse propagator appears in the numerator. The native checks include these cases. When $a \leq 0$ or $c \leq 0$, the remaining loop denominators carry only a lightlike external momentum. Tensor expansion of the polynomial numerator then leaves scaleless integrals, which vanish in dimensional regularization [1].

For $a, c \geq 1$, set $\Delta = A - 2$ and divide Eq. (76) by its value at $(1, 0, 1)$:

$$\begin{aligned}\frac{T(a, b, c)}{\mathcal{B}(s, d)} = \frac{s^{-\Delta}}{(a-1)!(c-1)!} &\frac{\Gamma(2 - d/2 + \Delta)}{\Gamma(2 - d/2)} \frac{\Gamma(d/2 - a - b)}{\Gamma(d/2 - 1)} \\ &\times \frac{\Gamma(d/2 - b - c)}{\Gamma(d/2 - 1)} \frac{\Gamma(d - 2)}{\Gamma(d - 2 - \Delta)}. \end{aligned} \tag{77}$$

The integer power Δ makes the signs from $(-1)^\Delta$ and $(-s)^{-\Delta}$ cancel. Every gamma-function argument differs from its denominator argument by an integer. Repeated application of $\Gamma(z+1) = z\Gamma(z)$, used in Appendix N, therefore gives

$$G(z, n) = \frac{\Gamma(z + n)}{\Gamma(z)} = \begin{cases} \displaystyle\prod_{j=0}^{n-1} (z + j), & n \geq 0, \\ \displaystyle\left[\prod_{j=1}^{-n} (z - j)\right]^{-1}, & n < 0. \end{cases} \tag{78}$$

The empty product equals one. These finite products express every tested triangle coefficient as a rational function of d and s. The method needs no seed prediction or elimination sequence within this domain. Independent loop blocks use the factorization derived earlier in the paper.

The learned decision concerns the native reductions that remain. For a target set $\mathcal{T}$, we compare the preceding application, two shared sparse seed regions, groups with equal numerator rank, and separate target reductions. All choices use exact IBP relations. A shared reduction solves the common equations once and reads the rows for every target. If

$$I_{\mathbf{a}} = \sum_m r_{\mathbf{a}m} M_m \quad (\mathbf{a} \in \mathcal{T}), \qquad \mathbf{I}_{\mathcal{T}} = R_{\mathcal{T}} \mathbf{M}, \tag{79}$$

then selecting several rows of $R_{\mathcal{T}}$ requires the same master basis $\mathbf{M}$. Sharing the equations changes the work needed to construct these rows; it preserves each reduction identity.

The sparse regions extend the earlier seed rule to a union over the requested targets. In a sector S, let $a_j \le 0$ denote a seed numerator power at an absent denominator. Its distance from a target's numerator box is

$$D_S(\mathbf{a},\mathbf{t}) = \sum_{j\notin S} \max(0, -a_j - \max(0, -t_j)) \,. \tag{80}$$

The shared region retains a seed when the smallest distance over $\mathbf{t} \in \mathcal{T}$ lies within the selected radius. It also keeps admissible target and master seeds and applies the native rank and propagator bounds. The two tested radii and bound parameters are fixed before collection. If the selected seed set does not complete the requested reduction, the existing recovery procedure expands the native calculation.

We train the policy from complete measured time. Let C_{ia} be the measured cost of choice a on training request i, including the fresh process, reduction, recovery, and exact coefficient comparison. An unsuccessful result receives a cost of at least 250 seconds. A leaf with request set S predicts

$$\widehat{C}_a(S) = \frac{1}{|S|} \sum_{i\in S} C_{ia}. \tag{81}$$

If a split creates S_L and S_R, the least recorded cost available to its two leaves is

$$\mathcal{L}(S_L, S_R) = \min_a \sum_{i\in S_L} C_{ia} + \min_a \sum_{i\in S_R} C_{ia}. \tag{82}$$

This follows by choosing one action independently in each leaf. We use this cost-sensitive form of recursive tree construction and also test bootstrap ensembles [10, 11]. Prediction averages the action costs across the selected trees. Validation selects the model, its margin over a fixed choice, and whether the complete application should enable it.

The model receives seventeen structural quantities: loop count, external-momentum count, total denominator count, physical denominator count, target count, minimum, maximum, and mean numerator rank, number of distinct ranks, maximum number of extra positive propagator powers, minimum active-line count, largest numerator power, maximum and mean pairwise numerator-index distance, largest number of active numerator positions, number of propagators that contain both loop momenta, and largest loop-momentum incidence count. The pairwise distance is the sum of absolute differences over auxiliary indices. These quantities describe the requested algebra and do not contain reference coefficients or family names. The measured learning domain consists of two-loop, seven-line, nine-index requests with numerator rank one through four. The independent-product tests through thirteen loops assess the exact method separately.

Q.1 Training inputs, controls, and basis preparation

The native learning data use the planar and nonplanar massless boxes defined in Appendix D. Their first seven indices equal one. We write the last two indices as $(-u, -v)$ and use these eight target collections for each graph and parameter profile:

$$\begin{aligned}
&\{(2,0)\}, \quad \{(0,3)\}, \quad \{(2,1)\}, \quad \{(2,0),(0,2)\},\\
&\{(3,0),(0,3),(2,1)\}, \quad \{(2,0),(0,3)\},\\
&\{(1,0),(2,0),(3,0),(4,0)\}, \quad \{(1,1),(2,1),(1,2),(2,2)\}.
\end{aligned}$$

Profiles 3011 and 3019 supply 32 training requests, while profile 3037 supplies sixteen validation requests. For profile h, the planar invariants are $s = -1009 - 12h$ and $t = -1021 - 14h$. The nonplanar convention uses $s = -113 - 18h$ and $t = -127 - 10h$, with the scalar products defined in Appendix D. Each training action runs once and each validation action runs twice. Candidate trees have depths one through four and minimum leaf populations of two, four, or eight. Ensembles

contain sixteen bootstrap trees, and a single tree provides the other candidate. We test margins of zero, three, and eight percent against the validation-selected fixed action. A further three-repeat, fresh-process validation at the new profile 3041 compares the selected model with the fixed action. We assign this validation and the reserved tests to the same compute node before they run. An interrupted validation on heterogeneous workers remains in the notes and does not determine admission. The model enters deployment only if it passes all exact checks, saves at least two percent in this full application comparison, and has an upper paired request-resampling ratio below one. All these decisions precede the reserved test.

The two sparse choices use the boundary vectors $(2.5, 0, 0, 1.25, 0, 1, -0.25)$ and $(1, 0, 0, 0, 1, 1, 0)$ in the earlier sparse-radius definition. Both use the native subsector rank truncation with one additional numerator rank. Rank grouping uses the second vector within each group. Single-target reduction and the preceding-application action preserve the earlier validated choices. The first four physical and structural feature counts are constant within this learning set, so their presence does not establish extrapolation in those quantities.

During development, repeated master discovery caused some planar batches to exceed the application limit. We therefore prepare an additional master-index registry. For three planar target collections and the rational and symbolic triangles, two native reductions with different integral orderings must agree exactly with each other and with the development reference. The planar collections must give the same canonical master-index set. The registry stores those indices and supplies no reduction coefficients. It can therefore remove repeated basis discovery while still requiring a new reduction at each kinematic point. Its preparation time is reported separately. Agreement on these checks establishes consistency for the tested inputs; it does not prove that a finite seed region finds the minimal basis for every kinematic specialization.

The reserved native test uses profiles 3053 and 3061, giving 32 requests from the original target collections. Eight further requests use profile 3067 and the new collections

$$\{(1,2)\}, \qquad \{(3,1)\}, \qquad \{(2,0),(1,1),(0,2)\}, \qquad \{(0,1),(1,1),(1,2),(1,3)\}.$$

Both graphs receive each new collection. These are new index batches within two known graphs. They do not test a previously unseen connected topology.

The exact tests contain one independent triangle product at every loop order $L = 1, \dots, 13$. Block $j = 0, \dots, L-1$ has powers

$$(a_j, b_j, c_j) = \big(1 + (j \bmod 3),\ b_j,\ 1 + (j \bmod 2)\big), \qquad b_j = \begin{cases} -1, & j \bmod 3 = 0, \\ 1, & \text{otherwise.} \end{cases}$$

The blocks use independent loop and external momenta, with a common rational scale $s = -101 - 6h$ at profile $h = 3071$. Some blocks have identical powers and kinematics, so the implementation can share their intermediate coefficients within the same request. A further one-loop triangle keeps s symbolic at profile 3079. Four compatibility requests use profile 3083: a thirteen-loop one-mass vacuum, a thirteen-loop massless propagator, a two-loop massive propagator, and a two-loop ladder. These use the previous family definitions. The entire reserved set therefore contains 58 requests.

Each request compares the final method, the same method with a fixed batch action, the same method with both learned choices disabled, the preceding method supplied with the new checked basis, and the unmodified preceding method. The same-basis control isolates the new analytic and learned choices from basis preparation. The unmodified control measures the practical change from the preceding application. Three repetitions rotate the method order. If the first repetition does not pass its independent check, later repetitions of that request and method are omitted. That outcome remains a failure and contributes no completed-time speedup. All comparisons retain the same target and reference coefficients. Native references use the independent ordering; triangle references require agreement between two native orderings.

For a method m and request i, let t_{imr} be complete time in repetition r. We first compute $\widetilde{t}_{im} = \operatorname{median}_r t_{imr}$ and then sum over requests verified by both compared methods. If these totals are T_{new} and T_{control}, their time ratio, speed factor, and percentage saving follow directly from

$$R = \frac{T_{\text{new}}}{T_{\text{control}}}, \qquad S = \frac{1}{R}, \qquad P = 100(1 - R).$$

The result figures use repeated resampling of the three paired repetitions within each fixed request. These intervals describe repetition variation on this test set. They do not quantify uncertainty over all integral families. The deployment gate instead resamples whole validation requests, as stated above.

References

[1] Stefan Weinzierl. *Feynman Integrals: A Comprehensive Treatment for Students and Researchers*. UNITEXT for Physics. Springer, 2022. arXiv:2201.03593; publisher DOI.

[2] K. G. Chetyrkin and F. V. Tkachov. Integration by parts: The algorithm to calculate β-functions in 4 loops. *Nuclear Physics B*, 192:159–204, 1981. publisher DOI.

[3] Philipp Maierhöfer, Johann Usovitsch, and Peter Uwer. Kira—a Feynman integral reduction program. *Computer Physics Communications*, 230:99–112, 2018. arXiv:1705.05610.

[4] Stefano Laporta. High-precision calculation of multiloop Feynman integrals by difference equations. *International Journal of Modern Physics A*, 15:5087–5159, 2000. arXiv:hep-ph/0102033.

[5] Fabian Lange, Johann Usovitsch, and Zihao Wu. Kira 3: Integral reduction with efficient seeding and optimized equation selection. *Computer Physics Communications*, 322:109999, 2026. arXiv:2505.20197.

[6] Zhuo-Yang Song, Tong-Zhi Yang, Qing-Hong Cao, Ming-xing Luo, and Hua Xing Zhu. Explainable AI-assisted optimization for Feynman integral reduction, 2025. arXiv:2502.09544.

[7] Justin Berman, Francois Charton, Andres Luna, Matthias Wilhelm, and Mao Zeng. Efficient ai-inspired reduction of feynman integrals via tube seeding, 2026. arXiv:2606.10698.

[8] Chang Hu, Bo Feng, Jiyuan Shen, and Yaobo Zhang. General one-loop generating function by integration-by-part relations. *Physical Review D*, 111:076026, 2025.

[9] Jonas Klappert, Fabian Lange, Philipp Maierhöfer, and Johann Usovitsch. Integral reduction with Kira 2.0 and finite field methods. *Computer Physics Communications*, 266:108024, 2021. arXiv:2008.06494.

[10] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Chapman & Hall, 1984. Publisher record.

[11] Leo Breiman. Bagging predictors. *Machine Learning*, 24:123–140, 1996. Publisher record.

[12] Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001.

[13] Pierre Geurts, Damien Ernst, and Louis Wehenkel. Extremely randomized trees. *Machine Learning*, 63:3–42, 2006.

[14] Fabian Pedregosa, Gael Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Edouard Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. Journal article.

[15] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. publisher DOI.

[16] Lin Xu, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. Hydra-mip: Automated algorithm configuration and selection for mixed integer programming. In *Proceedings of the 18th RCRA Workshop*, 2011.